

Studentų Sistema Vytenis Tumėnas
V3.0

Generated by Doxygen 1.17.0

1 Namespace Index	1
1.1 Namespace List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Namespace Documentation	9
5.1 my Namespace Reference	9
5.1.1 Typedef Documentation	9
5.1.1.1 Vector	9
6 Class Documentation	11
6.1 Studentas Class Reference	11
6.1.1 Detailed Description	12
6.1.2 Constructor & Destructor Documentation	12
6.1.2.1 Studentas() [1/3]	12
6.1.2.2 ~Studentas()	13
6.1.2.3 Studentas() [2/3]	13
6.1.2.4 Studentas() [3/3]	13
6.1.3 Member Function Documentation	13
6.1.3.1 addPaz()	13
6.1.3.2 clearPaz()	13
6.1.3.3 getEgz()	13
6.1.3.4 getMedrez()	13
6.1.3.5 getPaz()	14
6.1.3.6 getRez()	14
6.1.3.7 info()	14
6.1.3.8 operator=() [1/2]	14
6.1.3.9 operator=() [2/2]	14
6.1.3.10 refEgz()	14
6.1.3.11 refMedrez()	15
6.1.3.12 refPaz()	15
6.1.3.13 refRez()	15
6.1.3.14 setEgz()	15
6.1.3.15 skaiciuoti()	15
6.2 Vector< T, Allocator > Class Template Reference	15
6.2.1 Detailed Description	17
6.2.2 Member Typedef Documentation	17

6.2.2.1 allocator_type	17
6.2.2.2 const_iterator	17
6.2.2.3 const_pointer	17
6.2.2.4 const_reference	18
6.2.2.5 const_reverse_iterator	18
6.2.2.6 difference_type	18
6.2.2.7 iterator	18
6.2.2.8 pointer	18
6.2.2.9 reference	18
6.2.2.10 reverse_iterator	18
6.2.2.11 size_type	19
6.2.2.12 value_type	19
6.2.3 Constructor & Destructor Documentation	19
6.2.3.1 Vector() [1/6]	19
6.2.3.2 Vector() [2/6]	19
6.2.3.3 Vector() [3/6]	19
6.2.3.4 Vector() [4/6]	19
6.2.3.5 Vector() [5/6]	20
6.2.3.6 Vector() [6/6]	20
6.2.3.7 ~Vector()	20
6.2.4 Member Function Documentation	20
6.2.4.1 at() [1/2]	20
6.2.4.2 at() [2/2]	20
6.2.4.3 back() [1/2]	20
6.2.4.4 back() [2/2]	21
6.2.4.5 begin() [1/2]	21
6.2.4.6 begin() [2/2]	21
6.2.4.7 capacity()	21
6.2.4.8 cbegin()	21
6.2.4.9 cend()	21
6.2.4.10 clear()	21
6.2.4.11 data() [1/2]	22
6.2.4.12 data() [2/2]	22
6.2.4.13 emplace_back()	22
6.2.4.14 empty()	22
6.2.4.15 end() [1/2]	22
6.2.4.16 end() [2/2]	22
6.2.4.17 erase() [1/2]	23
6.2.4.18 erase() [2/2]	23
6.2.4.19 front() [1/2]	23
6.2.4.20 front() [2/2]	23
6.2.4.21 insert()	23

6.2.4.22 max_size()	23
6.2.4.23 operator"!=()	24
6.2.4.24 operator<()	24
6.2.4.25 operator<=()	24
6.2.4.26 operator=() [1/2]	24
6.2.4.27 operator=() [2/2]	24
6.2.4.28 operator==()	24
6.2.4.29 operator>()	25
6.2.4.30 operator>=()	25
6.2.4.31 operator[]() [1/2]	25
6.2.4.32 operator[]() [2/2]	25
6.2.4.33 pop_back()	25
6.2.4.34 push_back() [1/2]	25
6.2.4.35 push_back() [2/2]	26
6.2.4.36 rbegin() [1/2]	26
6.2.4.37 rbegin() [2/2]	26
6.2.4.38 rend() [1/2]	26
6.2.4.39 rend() [2/2]	26
6.2.4.40 reserve()	26
6.2.4.41 resize() [1/2]	27
6.2.4.42 resize() [2/2]	27
6.2.4.43 shrink_to_fit()	27
6.2.4.44 size()	27
6.2.4.45 swap()	27
6.3 Zmogus Class Reference	28
6.3.1 Detailed Description	28
6.3.2 Constructor & Destructor Documentation	29
6.3.2.1 Zmogus() [1/3]	29
6.3.2.2 ~Zmogus()	29
6.3.2.3 Zmogus() [2/3]	29
6.3.2.4 Zmogus() [3/3]	29
6.3.3 Member Function Documentation	29
6.3.3.1 getPavarde()	29
6.3.3.2 getVardas()	29
6.3.3.3 info()	30
6.3.3.4 operator=() [1/2]	30
6.3.3.5 operator=() [2/2]	30
6.3.3.6 refPavarde()	30
6.3.3.7 refVardas()	30
6.3.3.8 setPavarde()	30
6.3.3.9 setVardas()	30
6.3.4 Member Data Documentation	31

6.3.4.1 pavarde	31
6.3.4.2 vardas	31
7 File Documentation	33
7.1 funk.cpp File Reference	33
7.1.1 Function Documentation	34
7.1.1.1 cmpEgz()	34
7.1.1.2 cmpMedrez()	35
7.1.1.3 cmpPavarde()	35
7.1.1.4 cmpRez()	35
7.1.1.5 cmpVardas()	35
7.1.1.6 failgeneravimas()	35
7.1.1.7 failinputas()	36
7.1.1.8 failoutputas()	36
7.1.1.9 genirasymas()	36
7.1.1.10 inputas()	36
7.1.1.11 logResults()	36
7.1.1.12 lytgen()	37
7.1.1.13 matuoti_perskirstymus()	37
7.1.1.14 operator<<()	37
7.1.1.15 operator>>()	37
7.1.1.16 outputas()	37
7.1.1.17 randominputas()	38
7.1.1.18 randompavarde()	38
7.1.1.19 randomvardas()	38
7.1.1.20 rikiuoti()	38
7.1.1.21 skirstyti()	38
7.1.1.22 skirstyti2()	39
7.1.1.23 skirstyti3()	39
7.1.1.24 uzdmat()	39
7.2 funk.cpp	39
7.3 studentai.h File Reference	47
7.3.1 Enumeration Type Documentation	49
7.3.1.1 SkirstymoStrategija	49
7.3.2 Function Documentation	49
7.3.2.1 cmpEgz()	49
7.3.2.2 cmpMedrez()	49
7.3.2.3 cmpPavarde()	50
7.3.2.4 cmpRez()	50
7.3.2.5 cmpVardas()	50
7.3.2.6 failgeneravimas()	50
7.3.2.7 failinputas()	50

7.3.2.8 failoutputas()	50
7.3.2.9 genirasymas()	51
7.3.2.10 inputas()	51
7.3.2.11 logResults()	51
7.3.2.12 lytgen()	51
7.3.2.13 matuoti_perskirstymus()	51
7.3.2.14 operator<<()	51
7.3.2.15 operator>>()	52
7.3.2.16 outputas()	52
7.3.2.17 randominputas()	52
7.3.2.18 randompavarde()	52
7.3.2.19 randomvardas()	52
7.3.2.20 rikiuoti()	52
7.3.2.21 skirstyti()	53
7.3.2.22 skirstyti2()	53
7.3.2.23 skirstyti3()	53
7.3.2.24 uzdmat()	53
7.3.3 Variable Documentation	53
7.3.3.1 fgalunes	53
7.3.3.2 fvardai	53
7.3.3.3 irasymo_failas	53
7.3.3.4 irasymo_failas_blogas	54
7.3.3.5 irasymo_failas_geras	54
7.3.3.6 mgalunes	54
7.3.3.7 mvardai	54
7.3.3.8 pavardes	54
7.4 studentai.h	54
7.5 testing.cpp File Reference	57
7.5.1 Macro Definition Documentation	58
7.5.1.1 CATCH_CONFIG_MAIN	58
7.5.2 Function Documentation	58
7.5.2.1 makeStudent()	58
7.5.2.2 TEST_CASE() [1/37]	58
7.5.2.3 TEST_CASE() [2/37]	58
7.5.2.4 TEST_CASE() [3/37]	58
7.5.2.5 TEST_CASE() [4/37]	58
7.5.2.6 TEST_CASE() [5/37]	59
7.5.2.7 TEST_CASE() [6/37]	59
7.5.2.8 TEST_CASE() [7/37]	59
7.5.2.9 TEST_CASE() [8/37]	59
7.5.2.10 TEST_CASE() [9/37]	59
7.5.2.11 TEST_CASE() [10/37]	59

7.5.2.12 TEST_CASE() [11/37]	59
7.5.2.13 TEST_CASE() [12/37]	60
7.5.2.14 TEST_CASE() [13/37]	60
7.5.2.15 TEST_CASE() [14/37]	60
7.5.2.16 TEST_CASE() [15/37]	60
7.5.2.17 TEST_CASE() [16/37]	60
7.5.2.18 TEST_CASE() [17/37]	60
7.5.2.19 TEST_CASE() [18/37]	60
7.5.2.20 TEST_CASE() [19/37]	61
7.5.2.21 TEST_CASE() [20/37]	61
7.5.2.22 TEST_CASE() [21/37]	61
7.5.2.23 TEST_CASE() [22/37]	61
7.5.2.24 TEST_CASE() [23/37]	61
7.5.2.25 TEST_CASE() [24/37]	61
7.5.2.26 TEST_CASE() [25/37]	61
7.5.2.27 TEST_CASE() [26/37]	62
7.5.2.28 TEST_CASE() [27/37]	62
7.5.2.29 TEST_CASE() [28/37]	62
7.5.2.30 TEST_CASE() [29/37]	62
7.5.2.31 TEST_CASE() [30/37]	62
7.5.2.32 TEST_CASE() [31/37]	62
7.5.2.33 TEST_CASE() [32/37]	62
7.5.2.34 TEST_CASE() [33/37]	63
7.5.2.35 TEST_CASE() [34/37]	63
7.5.2.36 TEST_CASE() [35/37]	63
7.5.2.37 TEST_CASE() [36/37]	63
7.5.2.38 TEST_CASE() [37/37]	63
7.6 testing.cpp	64
7.7 Vector2.h File Reference	68
7.8 Vector2.h	69
7.9 versija_v3.0.cpp File Reference	75
7.9.1 Function Documentation	76
7.9.1.1 main()	76
7.10 versija_v3.0.cpp	76
Index	79

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

my	9
------------------------------	---

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Vector< T, Allocator >	15
Zmogus	28
Studentas	11

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas		
Studentas	klasė, paveldi iš Zmogus	11
Vector< T, Allocator >		15
Zmogus		
	Bazinė abstrakti klasė Zmogus	28

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

funk.cpp	33
studentai.h	47
testing.cpp	57
Vector2.h	68
versija_v3.0.cpp	75

Chapter 5

Namespace Documentation

5.1 my Namespace Reference

Typedefs

- `template<typename T>`
`using Vector = ::Vector<T>`

5.1.1 Typedef Documentation

5.1.1.1 Vector

```
template<typename T>
using my::Vector = ::Vector<T>
```

Definition at line [16](#) of file [studentai.h](#).

Chapter 6

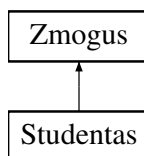
Class Documentation

6.1 Studentas Class Reference

`Studentas` klasė, paveldi iš `Zmogus`.

```
#include <studentai.h>
```

Inheritance diagram for `Studentas`:



Public Member Functions

- `Studentas` (const string &v="A", const string &p="BB", const `Vector`< int > &paz_={}, int egz_=0)
Konstruktorius su pažymiais ir egzaminu.
- `~Studentas` ()=default
- `Studentas` (const `Studentas` &other)
Kopijavimo konstruktorius.
- `Studentas` & `operator=` (const `Studentas` &other)
Kopijavimo operatorius.
- `Studentas` (`Studentas` &&other) noexcept
move konstruktorius
- `Studentas` & `operator=` (`Studentas` &&other) noexcept
move operatorius
- void `info` () const override
Abstrakti informacijos funkcija.
- `Vector`< int > & `refPaz` ()
- int & `refEgz` ()
- double & `refRez` ()
- double & `refMedrez` ()
- void `clearPaz` ()
- double `getRez` () const

- double [getMedrez](#) () const
- int [getEgz](#) () const
- const [Vector](#)< int > & [getPaz](#) () const
- void [addPaz](#) (int p)
- void [setEgz](#) (int e)
- void [skaiciuoti](#) ()

apskaičiuoja galutinius medianinius ir vidurkinius studento rezultatus

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) (const string &v="A", const string &p="BB")
Bazinis konstruktorius.
- virtual [~Zmogus](#) ()
Virtualus destruktorius.
- string & [refVardas](#) ()
- string & [refPavarde](#) ()
- string [getVardas](#) () const
- string [getPavarde](#) () const
- void [setVardas](#) (const string &v)
- void [setPavarde](#) (const string &p)
- [Zmogus](#) (const [Zmogus](#) &)=default
Rule of Zero (default) copy/move.
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &)=default
- [Zmogus](#) ([Zmogus](#) &&) noexcept=default
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&) noexcept=default

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string [vardas](#)
- string [pavarde](#)

6.1.1 Detailed Description

[Studentas](#) klasė, paveldi iš [Zmogus](#).

Definition at line 63 of file [studentai.h](#).

6.1.2 Constructor & Destructor Documentation

6.1.2.1 Studentas() [1/3]

```
Studentas::Studentas (
    const string & v = "A",
    const string & p = "BB",
    const Vector< int > & paz_ = {},
    int egz_ = 0) [inline]
```

Konstruktorius su pažymiais ir egzaminu.

Definition at line 72 of file [studentai.h](#).

6.1.2.2 ~Studentas()

```
Studentas::~~Studentas () [default]
```

6.1.2.3 Studentas() [2/3]

```
Studentas::Studentas (  
    const Studentas & other) [inline]
```

Kopijavimo konstruktorius.

Definition at line 93 of file [studentai.h](#).

6.1.2.4 Studentas() [3/3]

```
Studentas::Studentas (  
    Studentas && other) [inline], [noexcept]
```

move konstruktorius

Definition at line 114 of file [studentai.h](#).

6.1.3 Member Function Documentation

6.1.3.1 addPaz()

```
void Studentas::addPaz (  
    int p) [inline]
```

Definition at line 157 of file [studentai.h](#).

6.1.3.2 clearPaz()

```
void Studentas::clearPaz () [inline]
```

Definition at line 150 of file [studentai.h](#).

6.1.3.3 getEgz()

```
int Studentas::getEgz () const [inline]
```

Definition at line 153 of file [studentai.h](#).

6.1.3.4 getMedrez()

```
double Studentas::getMedrez () const [inline]
```

Definition at line 152 of file [studentai.h](#).

6.1.3.5 getPaz()

```
const Vector< int > & Studentas::getPaz () const [inline]
```

Definition at line 155 of file [studentai.h](#).

6.1.3.6 getRez()

```
double Studentas::getRez () const [inline]
```

Definition at line 151 of file [studentai.h](#).

6.1.3.7 info()

```
void Studentas::info () const [inline], [override], [virtual]
```

Abstrakti informacijos funkcija.

Implements [Zmogus](#).

Definition at line 142 of file [studentai.h](#).

6.1.3.8 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other) [inline]
```

Kopijavimo operatorius.

Definition at line 102 of file [studentai.h](#).

6.1.3.9 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [inline], [noexcept]
```

move operatorius

Definition at line 127 of file [studentai.h](#).

6.1.3.10 refEgz()

```
int & Studentas::refEgz () [inline]
```

Definition at line 147 of file [studentai.h](#).

6.1.3.11 refMedrez()

```
double & Studentas::refMedrez () [inline]
```

Definition at line 149 of file [studentai.h](#).

6.1.3.12 refPaz()

```
Vector< int > & Studentas::refPaz () [inline]
```

Definition at line 146 of file [studentai.h](#).

6.1.3.13 refRez()

```
double & Studentas::refRez () [inline]
```

Definition at line 148 of file [studentai.h](#).

6.1.3.14 setEgz()

```
void Studentas::setEgz (  
    int e) [inline]
```

Definition at line 158 of file [studentai.h](#).

6.1.3.15 skaiciuoti()

```
void Studentas::skaiciuoti ()
```

apskaičiuoja galutinius medianinius ir vidurkinius studento rezultatus

Compute final average and median-based results

Definition at line 18 of file [funk.cpp](#).

The documentation for this class was generated from the following files:

- [studentai.h](#)
- [funk.cpp](#)

6.2 Vector< T, Allocator > Class Template Reference

```
#include <Vector2.h>
```

Public Types

- using [value_type](#) = T
- using [allocator_type](#) = Allocator
- using [size_type](#) = std::size_t
- using [difference_type](#) = std::ptrdiff_t
- using [reference](#) = T&
- using [const_reference](#) = const T&
- using [pointer](#) = T*
- using [const_pointer](#) = const T*
- using [iterator](#) = T*
- using [const_iterator](#) = const T*
- using [reverse_iterator](#) = std::reverse_iterator<[iterator](#)>
- using [const_reverse_iterator](#) = std::reverse_iterator<[const_iterator](#)>

Public Member Functions

- [Vector](#) ()
- [Vector](#) ([size_type](#) count)
- [Vector](#) ([size_type](#) count, const T &value)
- [Vector](#) (std::initializer_list< T > init)
- [Vector](#) (const [Vector](#) &other)
- [Vector](#) ([Vector](#) &&other) noexcept
- [~Vector](#) ()
- [Vector](#) & [operator=](#) (const [Vector](#) &other)
- [Vector](#) & [operator=](#) ([Vector](#) &&other) noexcept
- [iterator](#) [begin](#) () noexcept
- [const_iterator](#) [begin](#) () const noexcept
- [const_iterator](#) [cbegin](#) () const noexcept
- [iterator](#) [end](#) () noexcept
- [const_iterator](#) [end](#) () const noexcept
- [const_iterator](#) [cend](#) () const noexcept
- [reverse_iterator](#) [rbegin](#) () noexcept
- [reverse_iterator](#) [rend](#) () noexcept
- [const_reverse_iterator](#) [rbegin](#) () const noexcept
- [const_reverse_iterator](#) [rend](#) () const noexcept
- bool [empty](#) () const noexcept
- [size_type](#) [size](#) () const noexcept
- [size_type](#) [capacity](#) () const noexcept
- [size_type](#) [max_size](#) () const noexcept
- void [reserve](#) ([size_type](#) new_capacity)
- void [shrink_to_fit](#) ()
- void [resize](#) ([size_type](#) count)
- void [resize](#) ([size_type](#) count, const T &value)
- [reference](#) [operator\[\]](#) ([size_type](#) index)
- [const_reference](#) [operator\[\]](#) ([size_type](#) index) const
- [reference](#) [at](#) ([size_type](#) index)
- [const_reference](#) [at](#) ([size_type](#) index) const
- [reference](#) [front](#) ()
- [const_reference](#) [front](#) () const
- [reference](#) [back](#) ()
- [const_reference](#) [back](#) () const
- [pointer](#) [data](#) () noexcept
- [const_pointer](#) [data](#) () const noexcept

- void [clear](#) () noexcept
- void [push_back](#) (const T &value)
- void [push_back](#) (T &&value)
- template<typename... Args>
 [reference](#) [emplace_back](#) (Args &&... args)
- void [pop_back](#) ()
- [iterator](#) [insert](#) ([iterator](#) pos, const T &value)
- [iterator](#) [erase](#) ([iterator](#) pos)
- void [swap](#) ([Vector](#) &other) noexcept
- bool [operator==](#) (const [Vector](#) &other) const
- bool [operator!=](#) (const [Vector](#) &other) const
- bool [operator<](#) (const [Vector](#) &other) const
- bool [operator>](#) (const [Vector](#) &other) const
- bool [operator<=](#) (const [Vector](#) &other) const
- bool [operator>=](#) (const [Vector](#) &other) const
- [iterator](#) [erase](#) ([iterator](#) first, [iterator](#) last)

6.2.1 Detailed Description

```
template<typename T, typename Allocator = std::allocator<T>>  
class Vector< T, Allocator >
```

Definition at line 13 of file [Vector2.h](#).

6.2.2 Member Typedef Documentation

6.2.2.1 allocator_type

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::allocator_type = Allocator
```

Definition at line 17 of file [Vector2.h](#).

6.2.2.2 const_iterator

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::const_iterator = const T*
```

Definition at line 26 of file [Vector2.h](#).

6.2.2.3 const_pointer

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::const_pointer = const T*
```

Definition at line 23 of file [Vector2.h](#).

6.2.2.4 const_reference

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_reference = const T&
```

Definition at line 21 of file [Vector2.h](#).

6.2.2.5 const_reverse_iterator

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

Definition at line 28 of file [Vector2.h](#).

6.2.2.6 difference_type

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::difference_type = std::ptrdiff_t
```

Definition at line 19 of file [Vector2.h](#).

6.2.2.7 iterator

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::iterator = T*
```

Definition at line 25 of file [Vector2.h](#).

6.2.2.8 pointer

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::pointer = T*
```

Definition at line 22 of file [Vector2.h](#).

6.2.2.9 reference

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::reference = T&
```

Definition at line 20 of file [Vector2.h](#).

6.2.2.10 reverse_iterator

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::reverse_iterator = std::reverse_iterator<iterator>
```

Definition at line 27 of file [Vector2.h](#).

6.2.2.11 size_type

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::size_type = std::size_t
```

Definition at line 18 of file [Vector2.h](#).

6.2.2.12 value_type

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::value_type = T
```

Definition at line 16 of file [Vector2.h](#).

6.2.3 Constructor & Destructor Documentation

6.2.3.1 Vector() [1/6]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector () [inline]
```

Definition at line 121 of file [Vector2.h](#).

6.2.3.2 Vector() [2/6]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    size_type count) [inline], [explicit]
```

Definition at line 128 of file [Vector2.h](#).

6.2.3.3 Vector() [3/6]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    size_type count,
    const T & value) [inline]
```

Definition at line 134 of file [Vector2.h](#).

6.2.3.4 Vector() [4/6]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    std::initializer_list< T > init) [inline]
```

Definition at line 143 of file [Vector2.h](#).

6.2.3.5 Vector() [5/6]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Vector< T, Allocator > & other) [inline]
```

Definition at line 152 of file [Vector2.h](#).

6.2.3.6 Vector() [6/6]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

Definition at line 161 of file [Vector2.h](#).

6.2.3.7 ~Vector()

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::~~Vector () [inline]
```

Definition at line 172 of file [Vector2.h](#).

6.2.4 Member Function Documentation

6.2.4.1 at() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::at (
    size_type index) [inline]
```

Definition at line 325 of file [Vector2.h](#).

6.2.4.2 at() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::at (
    size_type index) const [inline]
```

Definition at line 333 of file [Vector2.h](#).

6.2.4.3 back() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::back () [inline]
```

Definition at line 351 of file [Vector2.h](#).

6.2.4.4 back() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::back () const [inline]
```

Definition at line 356 of file [Vector2.h](#).

6.2.4.5 begin() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::begin () const [inline], [noexcept]
```

Definition at line 218 of file [Vector2.h](#).

6.2.4.6 begin() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::begin () [inline], [noexcept]
```

Definition at line 217 of file [Vector2.h](#).

6.2.4.7 capacity()

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::capacity () const [inline], [noexcept]
```

Definition at line 261 of file [Vector2.h](#).

6.2.4.8 cbegin()

```
template<typename T, typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::cbegin () const [inline], [noexcept]
```

Definition at line 220 of file [Vector2.h](#).

6.2.4.9 cend()

```
template<typename T, typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::cend () const [inline], [noexcept]
```

Definition at line 225 of file [Vector2.h](#).

6.2.4.10 clear()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::clear () [inline], [noexcept]
```

Definition at line 375 of file [Vector2.h](#).

6.2.4.11 data() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_pointer Vector< T, Allocator >::data () const [inline], [noexcept]
```

Definition at line 366 of file [Vector2.h](#).

6.2.4.12 data() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
pointer Vector< T, Allocator >::data () [inline], [noexcept]
```

Definition at line 361 of file [Vector2.h](#).

6.2.4.13 emplace_back()

```
template<typename T, typename Allocator = std::allocator<T>>
template<typename... Args>
reference Vector< T, Allocator >::emplace_back (
    Args &&... args) [inline]
```

Definition at line 407 of file [Vector2.h](#).

6.2.4.14 empty()

```
template<typename T, typename Allocator = std::allocator<T>>
bool Vector< T, Allocator >::empty () const [inline], [noexcept]
```

Definition at line 251 of file [Vector2.h](#).

6.2.4.15 end() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::end () const [inline], [noexcept]
```

Definition at line 223 of file [Vector2.h](#).

6.2.4.16 end() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::end () [inline], [noexcept]
```

Definition at line 222 of file [Vector2.h](#).

6.2.4.17 erase() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    iterator first,
    iterator last) [inline]
```

Definition at line 542 of file [Vector2.h](#).

6.2.4.18 erase() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    iterator pos) [inline]
```

Definition at line 468 of file [Vector2.h](#).

6.2.4.19 front() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::front () [inline]
```

Definition at line 341 of file [Vector2.h](#).

6.2.4.20 front() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::front () const [inline]
```

Definition at line 346 of file [Vector2.h](#).

6.2.4.21 insert()

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    iterator pos,
    const T & value) [inline]
```

Definition at line 435 of file [Vector2.h](#).

6.2.4.22 max_size()

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::max_size () const [inline], [noexcept]
```

Definition at line 266 of file [Vector2.h](#).

6.2.4.23 operator"!=()

```
template<typename T, typename Allocator = std::allocator<T>>
bool Vector< T, Allocator >::operator!= (
    const Vector< T, Allocator > & other) const [inline]
```

Definition at line 512 of file [Vector2.h](#).

6.2.4.24 operator<()

```
template<typename T, typename Allocator = std::allocator<T>>
bool Vector< T, Allocator >::operator< (
    const Vector< T, Allocator > & other) const [inline]
```

Definition at line 517 of file [Vector2.h](#).

6.2.4.25 operator<=()

```
template<typename T, typename Allocator = std::allocator<T>>
bool Vector< T, Allocator >::operator<= (
    const Vector< T, Allocator > & other) const [inline]
```

Definition at line 532 of file [Vector2.h](#).

6.2.4.26 operator=() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    const Vector< T, Allocator > & other) [inline]
```

Definition at line 182 of file [Vector2.h](#).

6.2.4.27 operator=() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

Definition at line 193 of file [Vector2.h](#).

6.2.4.28 operator==(())

```
template<typename T, typename Allocator = std::allocator<T>>
bool Vector< T, Allocator >::operator==(
    const Vector< T, Allocator > & other) const [inline]
```

Definition at line 498 of file [Vector2.h](#).

6.2.4.29 operator>()

```
template<typename T, typename Allocator = std::allocator<T>>
bool Vector< T, Allocator >::operator> (
    const Vector< T, Allocator > & other) const [inline]
```

Definition at line 527 of file [Vector2.h](#).

6.2.4.30 operator>=()

```
template<typename T, typename Allocator = std::allocator<T>>
bool Vector< T, Allocator >::operator>= (
    const Vector< T, Allocator > & other) const [inline]
```

Definition at line 537 of file [Vector2.h](#).

6.2.4.31 operator[]() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::operator[] (
    size_type index) [inline]
```

Definition at line 315 of file [Vector2.h](#).

6.2.4.32 operator[]() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::operator[] (
    size_type index) const [inline]
```

Definition at line 320 of file [Vector2.h](#).

6.2.4.33 pop_back()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::pop_back () [inline]
```

Definition at line 422 of file [Vector2.h](#).

6.2.4.34 push_back() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    const T & value) [inline]
```

Definition at line 380 of file [Vector2.h](#).

6.2.4.35 push_back() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    T && value) [inline]
```

Definition at line 393 of file [Vector2.h](#).

6.2.4.36 rbegin() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::rbegin () const [inline], [noexcept]
```

Definition at line 237 of file [Vector2.h](#).

6.2.4.37 rbegin() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reverse_iterator Vector< T, Allocator >::rbegin () [inline], [noexcept]
```

Definition at line 227 of file [Vector2.h](#).

6.2.4.38 rend() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::rend () const [inline], [noexcept]
```

Definition at line 242 of file [Vector2.h](#).

6.2.4.39 rend() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reverse_iterator Vector< T, Allocator >::rend () [inline], [noexcept]
```

Definition at line 232 of file [Vector2.h](#).

6.2.4.40 reserve()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::reserve (
    size_type new_capacity) [inline]
```

Definition at line 271 of file [Vector2.h](#).

6.2.4.41 `resize()` [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize (
    size_type count) [inline]
```

Definition at line 283 of file [Vector2.h](#).

6.2.4.42 `resize()` [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize (
    size_type count,
    const T & value) [inline]
```

Definition at line 288 of file [Vector2.h](#).

6.2.4.43 `shrink_to_fit()`

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::shrink_to_fit () [inline]
```

Definition at line 277 of file [Vector2.h](#).

6.2.4.44 `size()`

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::size () const [inline], [noexcept]
```

Definition at line 256 of file [Vector2.h](#).

6.2.4.45 `swap()`

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::swap (
    Vector< T, Allocator > & other) [inline], [noexcept]
```

Definition at line 486 of file [Vector2.h](#).

The documentation for this class was generated from the following file:

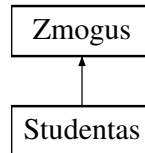
- [Vector2.h](#)

6.3 Zmogus Class Reference

Bazinė abstrakti klasė [Zmogus](#).

```
#include <studentai.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) (const string &v="A", const string &p="BB")
Bazinis konstruktorius.
- virtual [~Zmogus](#) ()
Virtualus destruktorius.
- virtual void [info](#) () const =0
Abstrakti informacijos funkcija.
- string & [refVardas](#) ()
- string & [refPavarde](#) ()
- string [getVardas](#) () const
- string [getPavarde](#) () const
- void [setVardas](#) (const string &v)
- void [setPavarde](#) (const string &p)
- [Zmogus](#) (const [Zmogus](#) &)=default
Rule of Zero (default) copy/move.
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &)=default
- [Zmogus](#) ([Zmogus](#) &&) noexcept=default
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&) noexcept=default

Protected Attributes

- string [vardas](#)
- string [pavarde](#)

6.3.1 Detailed Description

Bazinė abstrakti klasė [Zmogus](#).

Definition at line 32 of file [studentai.h](#).

6.3.2 Constructor & Destructor Documentation

6.3.2.1 Zmogus() [1/3]

```
Zmogus::Zmogus (
    const string & v = "A",
    const string & p = "BB") [inline]
```

Bazinis konstruktorius.

Definition at line 39 of file [studentai.h](#).

6.3.2.2 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

Virtualus destruktorius.

Definition at line 43 of file [studentai.h](#).

6.3.2.3 Zmogus() [2/3]

```
Zmogus::Zmogus (
    const Zmogus & ) [default]
```

Rule of Zero (default) copy/move.

6.3.2.4 Zmogus() [3/3]

```
Zmogus::Zmogus (
    Zmogus && ) [default], [noexcept]
```

6.3.3 Member Function Documentation

6.3.3.1 getPavarde()

```
string Zmogus::getPavarde () const [inline]
```

Definition at line 51 of file [studentai.h](#).

6.3.3.2 getVardas()

```
string Zmogus::getVardas () const [inline]
```

Definition at line 50 of file [studentai.h](#).

6.3.3.3 info()

```
virtual void Zmogus::info () const [pure virtual]
```

Abstrakti informacijos funkcija.

Implemented in [Studentas](#).

6.3.3.4 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & ) [default]
```

6.3.3.5 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && ) [default], [noexcept]
```

6.3.3.6 refPavarde()

```
string & Zmogus::refPavarde () [inline]
```

Definition at line [49](#) of file [studentai.h](#).

6.3.3.7 refVardas()

```
string & Zmogus::refVardas () [inline]
```

Definition at line [48](#) of file [studentai.h](#).

6.3.3.8 setPavarde()

```
void Zmogus::setPavarde (
    const string & p) [inline]
```

Definition at line [53](#) of file [studentai.h](#).

6.3.3.9 setVardas()

```
void Zmogus::setVardas (
    const string & v) [inline]
```

Definition at line [52](#) of file [studentai.h](#).

6.3.4 Member Data Documentation

6.3.4.1 pavarde

```
string Zmogus::pavarde [protected]
```

Definition at line 35 of file [studentai.h](#).

6.3.4.2 vardas

```
string Zmogus::vardas [protected]
```

Definition at line 34 of file [studentai.h](#).

The documentation for this class was generated from the following file:

- [studentai.h](#)

Chapter 7

File Documentation

7.1 funk.cpp File Reference

```
#include <iostream>
#include <vector>
#include <iomanip>
#include <string>
#include <cmath>
#include <algorithm>
#include <random>
#include <ctime>
#include <fstream>
#include <sstream>
#include <chrono>
#include "studentai.h"
#include <stdexcept>
#include "Vector2.h"
```

Functions

- void `inputas` (`my::Vector< Studentas >` &grupe)
rankinė studentų įvestis
- void `outputas` (const `my::Vector< Studentas >` &grupe)
atspausdina studentus į console
- void `failoutputas` (const `my::Vector< Studentas >` &grupe, string writing)
studentų spausdinimas į failą (be rikiavimo)
- void `randominputas` (`my::Vector< Studentas >` &grupe, bool ArGeneruotiVardus)
atsitiktinių studentų generavimas
- void `failinputas` (`my::Vector< Studentas >` &grupe, string reading)
studentų nuskaitymas iš failo
- string `randomvardas` (string lytis)
random vardo generavimas iš listo
- string `randompavarde` (string lytis)
random pavardės generavimas iš listo
- string `lytgen` ()
random lyties generavimas

- bool `cmpVardas` (const `Studentas` &a, const `Studentas` &b)
vardo palyginimas
- bool `cmpPavarde` (const `Studentas` &a, const `Studentas` &b)
pavardės palyginimas
- bool `cmpRez` (const `Studentas` &a, const `Studentas` &b)
rezultatų palyginimas
- bool `cmpMedrez` (const `Studentas` &a, const `Studentas` &b)
medianos rezultatų palyginimas
- bool `cmpEgz` (const `Studentas` &a, const `Studentas` &b)
egzamino rezultatų palyginimas
- void `rikiuoti` (`my::Vector`< `Studentas` > &grupe, int pasirinkimas)
studentų rikiavimas pagal cmp funkcijas
- void `failgeneravimas` ()
generuoja 5 failus su atsitiktiniais studentais
- void `genirasymas` (`my::Vector`< `Studentas` > &grupe, `my::Vector`< `Studentas` > &protingi, `my::Vector`< `Studentas` > &neprotingi, string writing_good, string writing_bad, int rik_pasirinkimas, `SkirstymoStrategija` strategija)
skaito gen failus iš failgeneravimas funkcijos, rikiuoja ir paskirsto į naujus failus
- void `skirstyti` (const `my::Vector`< `Studentas` > &grupe, `my::Vector`< `Studentas` > &protingi, `my::Vector`< `Studentas` > &neprotingi)
studentų skirstymas į grupes pagal rezultatus
- void `logResults` (const string &container, int size, double read_t, double sort_t, double split_t)
- void `skirstyti2` (`my::Vector`< `Studentas` > &grupe, `my::Vector`< `Studentas` > &neprotingi)
- void `skirstyti3` (`my::Vector`< `Studentas` > &grupe, `my::Vector`< `Studentas` > &neprotingi)
studentų skirstymas trečia taktika
- ostream & `operator<<` (ostream &os, const `Studentas` &s)
output operatorius klasei
- istream & `operator>>` (istream &is, `Studentas` &s)
input operatorius klasei
- void `uzdmat` (int vektordydis)
matuoja, kiek laiko uztrunka uzpildyti std::vector ir `Vector`
- void `matuoti_perskirstymus` (int kiekis)
Matuoja perskirstymus std::vector ir `Vector`.

7.1.1 Function Documentation

7.1.1.1 cmpEgz()

```
bool cmpEgz (
    const Studentas & a,
    const Studentas & b)
```

egzamino rezultatų palyginimas

Definition at line 364 of file `funk.cpp`.

7.1.1.2 cmpMedrez()

```
bool cmpMedrez (  
    const Studentas & a,  
    const Studentas & b)
```

medianos rezultatų palyginimas

Definition at line 359 of file [funk.cpp](#).

7.1.1.3 cmpPavarde()

```
bool cmpPavarde (  
    const Studentas & a,  
    const Studentas & b)
```

pavardės palyginimas

Definition at line 349 of file [funk.cpp](#).

7.1.1.4 cmpRez()

```
bool cmpRez (  
    const Studentas & a,  
    const Studentas & b)
```

rezultatų palyginimas

Definition at line 354 of file [funk.cpp](#).

7.1.1.5 cmpVardas()

```
bool cmpVardas (  
    const Studentas & a,  
    const Studentas & b)
```

vardo palyginimas

Definition at line 344 of file [funk.cpp](#).

7.1.1.6 failgeneravimas()

```
void failgeneravimas ()
```

generuoja 5 failus su atsitiktiniais studentais

Definition at line 385 of file [funk.cpp](#).

7.1.1.7 failinputas()

```
void failinputas (
    my::Vector< Studentas > & grupe,
    string reading)
```

studentų nuskaitymas iš failo

Definition at line 260 of file [funk.cpp](#).

7.1.1.8 failoutputas()

```
void failoutputas (
    const my::Vector< Studentas > & grupe,
    string writing)
```

studentų spausdinimas į failą (be rikiavimo)

Write students to file

Definition at line 142 of file [funk.cpp](#).

7.1.1.9 genirasymas()

```
void genirasymas (
    my::Vector< Studentas > & grupe,
    my::Vector< Studentas > & protingi,
    my::Vector< Studentas > & neprotingi,
    string writing_good,
    string writing_bad,
    int rik_pasirinkimas,
    SkirstymoStrategija strategija)
```

skaito gen failus iš failgeneravimas funkcijos, rikiuoja ir paskirsto į naujus failus

Definition at line 419 of file [funk.cpp](#).

7.1.1.10 inputas()

```
void inputas (
    my::Vector< Studentas > & grupe)
```

rankinė studentų įvestis

Definition at line 39 of file [funk.cpp](#).

7.1.1.11 logResults()

```
void logResults (
    const string & container,
    int size,
    double read_t,
    double sort_t,
    double split_t)
```

Definition at line 559 of file [funk.cpp](#).

7.1.1.12 lytgen()

```
string lytgen ()
```

random lyties generavimas

Definition at line 338 of file [funk.cpp](#).

7.1.1.13 matuoti_perskirstymus()

```
void matuoti_perskirstymus (  
    int kiekis)
```

Matuoja perskirstymus std::vector ir [Vector](#).

Definition at line 658 of file [funk.cpp](#).

7.1.1.14 operator<<()

```
ostream & operator<< (  
    ostream & os,  
    const Studentas & s)
```

output operatorius klasei

Definition at line 602 of file [funk.cpp](#).

7.1.1.15 operator>>()

```
istream & operator>> (  
    istream & is,  
    Studentas & s)
```

input operatorius klasei

Definition at line 611 of file [funk.cpp](#).

7.1.1.16 outputas()

```
void outputas (  
    const my::Vector< Studentas > & grupe)
```

atspausdina studentus į console

Print students to console

Definition at line 121 of file [funk.cpp](#).

7.1.1.17 randominputas()

```
void randominputas (
    my::Vector< Studentas > & grupe,
    bool ArGeneruotiVardus)
```

atsitiktinių studentų generavimas

Definition at line 167 of file [funk.cpp](#).

7.1.1.18 randompavarde()

```
string randompavarde (
    string lytis)
```

random pavardės generavimas iš listo

Definition at line 325 of file [funk.cpp](#).

7.1.1.19 randomvardas()

```
string randomvardas (
    string lytis)
```

random vardo generavimas iš listo

Definition at line 317 of file [funk.cpp](#).

7.1.1.20 rikiuoti()

```
void rikiuoti (
    my::Vector< Studentas > & grupe,
    int pasirinkimas)
```

studentų rikiavimas pagal cmp funkcijas

Definition at line 370 of file [funk.cpp](#).

7.1.1.21 skirstyti()

```
void skirstyti (
    const my::Vector< Studentas > & grupe,
    my::Vector< Studentas > & protingi,
    my::Vector< Studentas > & neprotingi)
```

studentų skirstymas į grupes pagal rezultatus

Definition at line 547 of file [funk.cpp](#).

7.1.1.22 skirstyti2()

```
void skirstyti2 (
    my::Vector< Studentas > & grupe,
    my::Vector< Studentas > & neprotingi)
```

Definition at line 571 of file [funk.cpp](#).

7.1.1.23 skirstyti3()

```
void skirstyti3 (
    my::Vector< Studentas > & grupe,
    my::Vector< Studentas > & neprotingi)
```

studentų skirstymas trečia taktika

Definition at line 584 of file [funk.cpp](#).

7.1.1.24 uzdmat()

```
void uzdmat (
    int vektordydis)
```

matuoja, kiek laiko uztrunka uzpildyti std::vector ir [Vector](#)

Definition at line 631 of file [funk.cpp](#).

7.2 funk.cpp

[Go to the documentation of this file.](#)

```
00001 #include <iostream>
00002 #include <vector>
00003 #include <iomanip>
00004 #include <string>
00005 #include <cmath>
00006 #include <algorithm>
00007 #include <random>
00008 #include <ctime>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <chrono>
00012 #include "studentai.h"
00013 #include <stdexcept>
00014 #include "Vector2.h"
00015
00016 using namespace std;
00017
00018 void Studentas::skaiciuoti()
00019 {
00020     if (paz.empty()) return;
00021     int sum = 0;
00022     for (int x : paz) sum += x;
00023     rez = 0.4 * (sum * 1.0 / paz.size()) + 0.6 * egz;
00024
00025     my::Vector<int> temp = paz;
00026     sort(temp.begin(), temp.end());
00027
00028     if (temp.size() % 2 == 0)
00029         medrez = (temp[temp.size()/2 - 1] + temp[temp.size()/2]) / 2.0;
00030     else
```

```

00034         medrez = temp[temp.size()/2];
00035
00036         medrez = medrez * 0.4 + egz * 0.6;
00037     }
00038
00039 void inputas(my::Vector<Studentas> &grupe)
00040 {
00041     Studentas A;
00042
00043     while (true)
00044     {
00045         int sum = 0;
00046
00047         cout << "Vardas ir pavarde: ";
00048         cin >> A.refVardas() >> A.refPavarde();
00049
00050         int i = 0;
00051         string ats;
00052         while (true)
00053         {
00054             int temp;
00055             cout << "Iveskite pazymi numeris " << i + 1 << ": ";
00056             try
00057             {
00058                 cin >> temp;
00059                 if (cin.fail())
00060                     throw runtime_error("Bloga ivestis.");
00061             }
00062             catch (exception& e)
00063             {
00064                 cin.clear();
00065                 cin.ignore(10000, '\n');
00066                 cout << e.what() << " Bandykite dar karta.\n";
00067                 continue;
00068             }
00069
00070             A.refPaz().push_back(temp);
00071             sum += temp;
00072
00073             cout << "Ar toliau vesite pazymius? (y/n): ";
00074             cin >> ats;
00075             if (ats == "n") break;
00076             else if (ats == "y") cout << "Veskite toliau: ";
00077
00078             i++;
00079         }
00080
00081         cout << "Iveskite egzamino rezultata: ";
00082         try
00083         {
00084             cin >> A.refEgz();
00085             if (cin.fail())
00086                 throw runtime_error("Bloga ivestis.");
00087         }
00088         catch (exception& e)
00089         {
00090             cin.clear();
00091             cin.ignore(10000, '\n');
00092             cout << e.what() << " Bandykite dar karta.\n";
00093             continue;
00094         }
00095
00096         if (!A.refPaz().empty())
00097         {
00098             A.refRez() = (sum * 1.0 / A.refPaz().size()) * 0.4 + A.refEgz() * 0.6;
00099
00100             sort(A.refPaz().begin(), A.refPaz().end());
00101
00102             int n = A.refPaz().size();
00103             if (n % 2 == 0)
00104                 A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00105             else
00106                 A.refMedrez() = A.refPaz()[n/2];
00107
00108             A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00109             A.skaiciuoti();
00110         }
00111
00112         grupe.push_back(A);
00113         A.clearPaz();
00114
00115         cout << "Ar toliau vesite mokinius? (y/n): ";
00116         cin >> ats;
00117         if (ats == "n") break;
00118     }
00119 }
00120

```



```

00121 void outputas(const my::Vector<Studentas> &grupe)
00122 {
00124     cout << "-----" << endl;
00125     cout << left << setw(20) << "Vardas"
00126         << setw(20) << "Pavarde"
00127         << setw(20) << "Rezultatas (Vid.) /"
00128         << setw(20) << " Rezultatas (Med.)"
00129         << endl;
00130     cout << "-----" << endl;
00131
00132     for (const auto &A : grupe)
00133     {
00134         cout << left << setw(20) << A.getVardas()
00135             << setw(20) << A.getPavarde()
00136             << right << setw(20) << fixed << setprecision(2) << A.getRez()
00137             << setw(20) << A.getMedrez()
00138             << endl;
00139     }
00140 }
00141
00142 void failoutputas(const my::Vector<Studentas> &grupe, string writing)
00143 {
00145     ofstream out(writing);
00146     out << "----- \n";
00147     out << left << setw(20) << "Vardas"
00148         << setw(20) << "Pavarde"
00149         << setw(20) << "Rezultatas (Vid.) /"
00150         << setw(20) << " Rezultatas (Med.)"
00151         << "\n";
00152     out << "----- \n";
00153
00154     for (const Studentas& s : grupe)
00155     {
00156         out << left << setw(20) << s.getVardas()
00157             << setw(20) << s.getPavarde()
00158             << fixed << setprecision(2)
00159             << setw(20) << s.getRez()
00160             << setw(20) << s.getMedrez()
00161             << '\n';
00162     }
00163
00164     cout << "Studentu skaicius: " << grupe.size() << endl;
00165 }
00166
00167 void randominputas(my::Vector<Studentas> &grupe, bool ArGeneruotiVardus)
00168 {
00169     Studentas A;
00170     string lytis;
00171     string ats;
00172
00173     if (ArGeneruotiVardus)
00174     {
00175         int zmonsk = rand() % 100 + 1;
00176
00177         for(int iii = 0; iii < zmonsk; iii++)
00178         {
00179             int sum = 0;
00180
00181             lytis = lytgen();
00182             A.refVardas() = randomvardas(lytis);
00183             A.refPavarde() = randompavarde(lytis);
00184
00185             int pazsk = rand() % 100 + 1;
00186
00187             for (int ii = 0; ii < pazsk; ii++)
00188             {
00189                 int temp = rand() % 10 + 1;
00190                 A.refPaz().push_back(temp);
00191                 sum += temp;
00192             }
00193
00194             A.refEgz() = rand() % 10 + 1;
00195
00196             if (!A.refPaz().empty())
00197             {
00198                 A.refRez() = (sum * 1.0 / A.refPaz().size()) * 0.4 + A.refEgz() * 0.6;
00199
00200                 sort(A.refPaz().begin(), A.refPaz().end());
00201
00202                 int n = A.refPaz().size();
00203                 if (n % 2 == 0)
00204                     A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00205                 else
00206                     A.refMedrez() = A.refPaz()[n/2];
00207
00208                 A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00209             }

```

```

00210
00211     grupe.push_back(A);
00212     A.clearPaz();
00213 }
00214 }
00215 else
00216 {
00217     while (true)
00218     {
00219         int sum = 0;
00220
00221         cout << "Vardas ir pavarde: ";
00222         cin >> A.refVardas() >> A.refPavarde();
00223
00224         int pazsk = rand() % 100 + 1;
00225
00226         for (int i = 0; i < pazsk; i++)
00227         {
00228             int temp = rand() % 10 + 1;
00229             A.refPaz().push_back(temp);
00230             sum += temp;
00231         }
00232
00233         A.refEgz() = rand() % 10 + 1;
00234
00235         if (!A.refPaz().empty())
00236         {
00237             A.refRez() = (sum * 1.0 / A.refPaz().size()) * 0.4 + A.refEgz() * 0.6;
00238
00239             sort(A.refPaz().begin(), A.refPaz().end());
00240
00241             int n = A.refPaz().size();
00242             if (n % 2 == 0)
00243                 A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00244             else
00245                 A.refMedrez() = A.refPaz()[n/2];
00246
00247             A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00248         }
00249
00250         grupe.push_back(A);
00251         A.clearPaz();
00252
00253         cout << "Ar toliau vesite zmones? (y/n): ";
00254         cin >> ats;
00255         if (ats == "n") break;
00256     }
00257 }
00258 }
00259
00260 void failinputas(my::Vector<Studentas> &grupe, string reading)
00261 {
00262     ifstream file(reading);
00263     grupe.reserve(1000000);
00264
00265     if (!file)
00266     {
00267         cout << "Nepavyko atidaryti failo\n";
00268         return;
00269     }
00270
00271     string line;
00272     getline(file, line);
00273
00274     while (getline(file, line))
00275     {
00276         istringstream iss(line);
00277
00278         Studentas A;
00279
00280         iss >> A.refVardas() >> A.refPavarde();
00281
00282         int pazmys;
00283
00284         while (iss >> pazmys)
00285             A.refPaz().push_back(pazmys);
00286
00287         if (!A.refPaz().empty())
00288         {
00289             A.refEgz() = A.refPaz().back();
00290             A.refPaz().pop_back();
00291         }
00292
00293         if (!A.refPaz().empty())
00294         {
00295             int sum = 0;
00296             for (int x : A.refPaz())

```

```

00297         sum += x;
00298
00299         sort(A.refPaz().begin(), A.refPaz().end());
00300
00301         int n = A.refPaz().size();
00302
00303         if (n % 2 == 0)
00304             A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00305         else
00306             A.refMedrez() = A.refPaz()[n/2];
00307
00308         A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00309         A.refRez() = 0.4 * (sum * 1.0 / A.refPaz().size()) + 0.6 * A.refEgz();
00310     }
00311
00312     grupe.push_back(A);
00313 }
00314 }
00315
00316
00317 string randomvardas(string lytis)
00318 {
00319     if (lytis == "mot")
00320         return fvardai[rand() % fvardai.size()];
00321     else
00322         return mvardai[rand() % mvardai.size()];
00323 }
00324
00325 string randompavarde(string lytis)
00326 {
00327     int index = rand() % pavardes.size();
00328     string galune;
00329
00330     if (lytis == "mot")
00331         galune = fgalunes[rand() % fgalunes.size()];
00332     else
00333         galune = mgalunes[rand() % mgalunes.size()];
00334
00335     return pavardes[index] + galune;
00336 }
00337
00338 string lytgen()
00339 {
00340     return rand() % 2 == 0 ? "vyr" : "mot";
00341 }
00342
00343
00344 bool cmpVardas(const Studentas &a, const Studentas &b)
00345 {
00346     return a.getVardas() < b.getVardas();
00347 }
00348
00349 bool cmpPavarde(const Studentas &a, const Studentas &b)
00350 {
00351     return a.getPavarde() < b.getPavarde();
00352 }
00353
00354 bool cmpRez(const Studentas &a, const Studentas &b)
00355 {
00356     return a.getRez() < b.getRez();
00357 }
00358
00359 bool cmpMedrez(const Studentas &a, const Studentas &b)
00360 {
00361     return a.getMedrez() < b.getMedrez();
00362 }
00363
00364 bool cmpEgz(const Studentas &a, const Studentas &b)
00365 {
00366     return a.getEgz() < b.getEgz();
00367 }
00368
00369
00370 void rikiuoti(my::Vector<Studentas> &grupe, int pasirinkimas)
00371 {
00372     if (pasirinkimas == 1)
00373         sort(grupe.begin(), grupe.end(), cmpVardas);
00374     else if (pasirinkimas == 2)
00375         sort(grupe.begin(), grupe.end(), cmpPavarde);
00376     else if (pasirinkimas == 3)
00377         sort(grupe.begin(), grupe.end(), cmpRez);
00378     else if (pasirinkimas == 4)
00379         sort(grupe.begin(), grupe.end(), cmpMedrez);
00380     else if (pasirinkimas == 5)
00381         sort(grupe.begin(), grupe.end(), cmpEgz);
00382 }
00383

```

```

00384
00385 void failgeneravimas()
00386 {
00387     int sk = 1000;
00388     for (int it = 0; it < 5; it++)
00389     {
00390         cout << "pradedamas " << sk << " eiluciu failo generavimo laiko skaiciavimas" << endl;
00391         auto input_start = chrono::high_resolution_clock::now();
00392
00393         ofstream gen("gen" + to_string(sk) + ".txt");
00394         gen << left << setw(20) << "STUD. VARDAS"
00395             << setw(20) << "STUD. PAVARDE"
00396             << setw(20) << "STUD PAZYMAI (PASKUTINIS EGZAMINO)\n";
00397
00398         for (int genit = 1; genit <= sk; genit++)
00399         {
00400             gen << left << setw(20) << "genVardas" + to_string(genit)
00401                 << setw(20) << "genPavarde" + to_string(genit);
00402
00403             for (int i = 1; i <= 16; i++)
00404                 gen << setw(20) << rand() % 10 + 1;
00405
00406             gen << "\n";
00407         }
00408
00409         gen.close();
00410         sk *= 10;
00411
00412         auto input_end = chrono::high_resolution_clock::now();
00413         chrono::duration<double> input_diff = input_end - input_start;
00414         cout << "Failu sukurimas + uzdarymas uztruko: " << input_diff.count() << " s \n";
00415     }
00416 }
00417
00418
00419 void generasymas(my::Vector<Studentas> &grupe, my::Vector<Studentas> &protingi, my::Vector<Studentas>
&neprotingi,
00420     string writing_good, string writing_bad, int rik_pasirinkimas, SkirstymoStrategija strategija)
00421 {
00422     int sk = 1000;
00423     grupe.reserve(10000000);
00424     protingi.reserve(10000000);
00425     neprotingi.reserve(10000000);
00426
00427     for (int it = 0; it < 5; it++)
00428     {
00429         string filename = "gen" + to_string(sk) + ".txt";
00430
00431         cout << "Pradedamas failo nuskaitymas: " << filename << endl;
00432         auto start_read = chrono::high_resolution_clock::now();
00433
00434         ifstream file(filename);
00435
00436         if (!file)
00437         {
00438             cout << "Nepavyko atidaryti failo\n";
00439             sk *= 10;
00440             continue;
00441         }
00442
00443         string line;
00444
00445         getline(file, line); // skip header
00446
00447         while (getline(file, line))
00448         {
00449             istringstream iss(line);
00450
00451             Studentas A;
00452
00453             iss >> A.refVardas() >> A.refPavarde();
00454
00455             int paz;
00456             while (iss >> paz)
00457                 A.refPaz().push_back(paz);
00458
00459             if (A.refPaz().empty())
00460                 continue;
00461
00462             A.refEgz() = A.refPaz().back();
00463             A.refPaz().pop_back();
00464
00465             int sum = 0;
00466             for (int x : A.refPaz())
00467                 sum += x;
00468
00469             A.refRez() = 0.4 * (sum * 1.0 / A.refPaz().size()) + 0.6 * A.refEgz();

```

```

00470
00471         sort(A.refPaz().begin(), A.refPaz().end());
00472
00473         int n = A.refPaz().size();
00474
00475         if (n % 2 == 0)
00476             A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00477         else
00478             A.refMedrez() = A.refPaz()[n/2];
00479
00480         A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00481
00482         grupe.push_back(A);
00483     }
00484
00485     file.close();
00486
00487     auto end_read = chrono::high_resolution_clock::now();
00488     chrono::duration<double> diff_read = end_read - start_read;
00489
00490     cout << "Failo nuskaitymas uztruko: " << diff_read.count() << " s\n";
00491
00492     auto start_rikiavimas = chrono::high_resolution_clock::now();
00493     rikiuoti(grupe, rik_pasirinkimas);
00494     auto end_rikiavimas = chrono::high_resolution_clock::now();
00495     chrono::duration<double> diff_rikiavimas = end_rikiavimas - start_rikiavimas;
00496
00497     cout << "Rikiavimas failui " << filename << " uztruko: " << diff_rikiavimas.count() << " s\n";
00498
00499     auto start_skirstymas = chrono::high_resolution_clock::now();
00500
00501     if (strategija == SkirstymoStrategija::PIRMAS)
00502         skirstyti(grupe, protingi, neprotingi);
00503     else if (strategija == SkirstymoStrategija::ANTRAS)
00504         skirstyti2(grupe, neprotingi);
00505     else if (strategija == SkirstymoStrategija::TRECIAS)
00506         skirstyti3(grupe, neprotingi);
00507
00508     auto end_skirstymas = chrono::high_resolution_clock::now();
00509     chrono::duration<double> diff_skirstymas = end_skirstymas - start_skirstymas;
00510
00511     cout << "Protingu/Neprotingu skirstymas failui " << filename << " uztruko: " <<
diff_skirstymas.count() << " s\n";
00512
00513     if (strategija == SkirstymoStrategija::PIRMAS)
00514     {
00515         cout << "Protingu ";
00516         failoutputas(protingi, "kursiokai_geri_" + to_string(sk) + ".txt");
00517         cout << "Neprotingu ";
00518         failoutputas(neprotingi, "kursiokai_blogi_" + to_string(sk) + ".txt");
00519     }
00520     else
00521     {
00522         cout << "Protingu ";
00523         failoutputas(grupe, "kursiokai_geri_" + to_string(sk) + ".txt");
00524         cout << "Neprotingu ";
00525         failoutputas(neprotingi, "kursiokai_blogi_" + to_string(sk) + ".txt");
00526     }
00527
00528     cout << "\n";
00529
00530     grupe.clear();
00531     protingi.clear();
00532     neprotingi.clear();
00533
00534     logResults(
00535         "my::Vector",
00536         sk,
00537         diff_read.count(),
00538         diff_rikiavimas.count(),
00539         diff_skirstymas.count()
00540     );
00541
00542     sk *= 10;
00543 }
00544 }
00545
00546
00547 void skirstyti(const my::Vector<Studentas>& grupe, my::Vector<Studentas>& protingi,
my::Vector<Studentas>& neprotingi)
00548 {
00549     for (const auto& s : grupe)
00550     {
00551         if (s.getRez() >= 5.0)
00552             protingi.push_back(s);
00553         else
00554             neprotingi.push_back(s);

```

```

00555     }
00556 }
00557
00558
00559 void logResults(const string& container, int size,
00560               double read_t, double sort_t, double split_t)
00561 {
00562     ofstream out("results.csv", ios::app);
00563     out << container << ", "
00564         << size << ", "
00565         << read_t << ", "
00566         << sort_t << ", "
00567         << split_t << "\n";
00568 }
00569
00570
00571 void skirstyti2(my::Vector<Studentas>& grupe, my::Vector<Studentas>& neprotingi)
00572 {
00573     auto it = partition(grupe.begin(), grupe.end(),
00574                       [](const Studentas& s) { return s.getRez() >= 5.0; });
00575     for (auto iter = it; iter != grupe.end(); ++iter)
00576         neprotingi.push_back(*iter);
00577     grupe.erase(it, grupe.end());
00578 }
00579
00580
00581
00582
00583
00584 void skirstyti3(my::Vector<Studentas>& grupe, my::Vector<Studentas>& neprotingi)
00585 {
00586     neprotingi.reserve(grupe.size());
00587     auto it = remove_if(grupe.begin(), grupe.end(),
00588                       [&](const Studentas& s)
00589                       {
00590                           if (s.getRez() < 5.0)
00591                           {
00592                               neprotingi.push_back(s);
00593                               return true;
00594                           }
00595                           return false;
00596                       });
00597     grupe.erase(it, grupe.end());
00598 }
00599
00600
00601
00602 ostream& operator<<(ostream& os, const Studentas& s)
00603 {
00604     os << left << setw(20) << s.getVardas()
00605         << setw(20) << s.getPavarde()
00606         << setw(10) << fixed << setprecision(2) << s.getRez()
00607         << setw(10) << s.getMedrez();
00608     return os;
00609 }
00610
00611 istream& operator>>(istream& is, Studentas& s)
00612 {
00613     s.refPaz().clear();
00614     is >> s.refVardas() >> s.refPavarde();
00615     int paz;
00616     while (is >> paz) {
00617         s.refPaz().push_back(paz);
00618     }
00619     if (!s.refPaz().empty()) {
00620         s.refEgz() = s.refPaz().back();
00621         s.refPaz().pop_back();
00622         s.skaiciuoti();
00623     }
00624     return is;
00625 }
00626
00627
00628
00629
00630
00631 void uzdmat(int vektordydis)
00632 {
00633     cout << "matuojamas laikas vectoriaus dydziui: " << vektordydis << endl;
00634     // std::vector laiko matavimas
00635     auto start = chrono::high_resolution_clock::now();
00636     std::vector<int> v1;
00637     v1.reserve(vektordydis);
00638     for (int i = 1; i <= vektordydis; ++i) v1.push_back(i);
00639     auto end = chrono::high_resolution_clock::now();
00640     chrono::duration<double> diff = end - start;
00641

```

```

00642     cout << "std::vector push_back laikas: " << diff.count() << " s\n";
00643
00644     // custom Vector laiko matavimas
00645     auto start2 = chrono::high_resolution_clock::now();
00646     Vector<int> v2;
00647     v2.reserve(vektordydis);
00648     for (int i = 1; i <= vektordydis; ++i) v2.push_back(i);
00649     auto end2 = chrono::high_resolution_clock::now();
00650     chrono::duration<double> diff2 = end2 - start2;
00651     cout << "custom Vector push_back laikas: " << diff2.count() << " s\n";
00652
00653     // CSV outputas
00654     ofstream out("results_matavimai.csv", ios::app);
00655     out << vektordydis << "," << diff.count() << "," << diff2.count() << "\n";
00656 }
00657
00658 void matuoti_perskirstymus(int kiekis)
00659 {
00660     cout << "\nMatuojami perskirstymai dydziui: " << kiekis << endl;
00661
00662     // --- std::vector ---
00663     {
00664         std::vector<int> v;
00665         int realloc_count = 0;
00666         size_t last_capacity = v.capacity();
00667
00668         for (int i = 0; i < kiekis; i++)
00669         {
00670             v.push_back(i);
00671             if (v.capacity() != last_capacity)
00672             {
00673                 realloc_count++;
00674                 last_capacity = v.capacity();
00675             }
00676         }
00677
00678         cout << "std::vector perskirstymu skaicius: " << realloc_count << endl;
00679     }
00680
00681     // --- custom Vector ---
00682     {
00683         Vector<int> v;
00684         int realloc_count = 0;
00685         size_t last_capacity = v.capacity();
00686
00687         for (int i = 0; i < kiekis; i++)
00688         {
00689             v.push_back(i);
00690             if (v.capacity() != last_capacity)
00691             {
00692                 realloc_count++;
00693                 last_capacity = v.capacity();
00694             }
00695         }
00696
00697         cout << "Custom Vector perskirstymu skaicius: " << realloc_count << endl;
00698     }
00699 }

```

7.3 studentai.h File Reference

```

#include <iostream>
#include <string>
#include <algorithm>
#include "Vector2.h"

```

Classes

- class [Zmogus](#)
Bazinė abstrakti klasė Zmogus.
- class [Studentas](#)
Studentas klasė, paveldi iš Zmogus.

Namespaces

- namespace [my](#)

Typedefs

- `template<typename T>`
using [my::Vector](#) = `::Vector<T>`

Enumerations

- enum class [SkirstymoStrategija](#) { [PIRMAS](#) , [ANTRAS](#) , [TRECIAS](#) }
- Strategijos studentų skirstymui.*

Functions

- void [inputas](#) ([Vector](#)< [Studentas](#) > &grupe)
- void [failinputas](#) ([Vector](#)< [Studentas](#) > &grupe, string reading)
- void [randominputas](#) ([Vector](#)< [Studentas](#) > &grupe, bool ArGeneruotiVardus)
- void [outputas](#) (const [Vector](#)< [Studentas](#) > &grupe)
- void [failoutputas](#) (const [Vector](#)< [Studentas](#) > &grupe, string writing)
- void [failgeneravimas](#) ()
generuoja 5 failus su atsitiktiniais studentais
- void [genirasymas](#) ([Vector](#)< [Studentas](#) > &grupe, [Vector](#)< [Studentas](#) > &protingi, [Vector](#)< [Studentas](#) > &neprotingi, string writing_good, string writing_bad, int rik_pasirinkimas, [SkirstymoStrategija](#) strategija)
- std::string [lytgen](#) ()
random lyties generavimas
- string [randomvardas](#) (string lytis)
random vardo generavimas iš listo
- string [randompavarde](#) (string lytis)
random pavardės generavimas iš listo
- void [rikiuoti](#) ([Vector](#)< [Studentas](#) > &grupe, int pasirinkimas)
- bool [cmpVardas](#) (const [Studentas](#) &a, const [Studentas](#) &b)
vardo palyginimas
- bool [cmpPavarde](#) (const [Studentas](#) &a, const [Studentas](#) &b)
pavardės palyginimas
- bool [cmpRez](#) (const [Studentas](#) &a, const [Studentas](#) &b)
rezultatų palyginimas
- bool [cmpMedrez](#) (const [Studentas](#) &a, const [Studentas](#) &b)
medianos rezultatų palyginimas
- bool [cmpEgz](#) (const [Studentas](#) &a, const [Studentas](#) &b)
egzamino rezultatų palyginimas
- void [skirstyti](#) (const [Vector](#)< [Studentas](#) > &grupe, [Vector](#)< [Studentas](#) > &protingi, [Vector](#)< [Studentas](#) > &neprotingi)
- void [logResults](#) (const string &container, int size, double read_t, double sort_t, double split_t)
- void [skirstyti2](#) ([Vector](#)< [Studentas](#) > &grupe, [Vector](#)< [Studentas](#) > &neprotingi)
- void [skirstyti3](#) ([Vector](#)< [Studentas](#) > &grupe, [Vector](#)< [Studentas](#) > &neprotingi)
- void [uzdmat](#) (int vektordydis)
matuoja, kiek laiko uztrunka uzpildyti std::vector ir [Vector](#)
- void [matuoti_perskirstymus](#) (int kiekis)
Matuoja perskirstymus std::vector ir [Vector](#).
- ostream & [operator<<](#) (ostream &os, const [Studentas](#) &s)
output operatorius klasei
- istream & [operator>>](#) (istream &is, [Studentas](#) &s)
input operatorius klasei

Variables

- const [Vector](#)< string > [mvardai](#) = {"Vytenis", "Tomas", "Jonas", "Matas", "Simas", "Mantas", "Arnas"}
- const [Vector](#)< string > [fvardai](#) = {"Egle", "Viktorija", "Vakare", "Inga", "Ema", "Marija", "Janina"}
- const [Vector](#)< string > [pavardes](#) = {"Macerausk", "Jankausk", "Kazlausk", "Svilpausk", "Drugeliausk", "Briedausk"}
- const [Vector](#)< string > [mgalunes](#) = {"as", "aitis"}
- const [Vector](#)< string > [fgalunes](#) = {"iene", "aite", "yte"}
- const string [irasymo_failas](#) = "kursiokai.txt"
- const string [irasymo_failas_geras](#) = "kursiokai_geri.txt"
- const string [irasymo_failas_blogas](#) = "kursiokai_blogi.txt"

7.3.1 Enumeration Type Documentation

7.3.1.1 SkirstymoStrategija

```
enum class SkirstymoStrategija [strong]
```

Strategijos studentų skirstymui.

Enumerator

PIRMAS	
ANTRAS	
TRECIAS	

Definition at line [25](#) of file [studentai.h](#).

7.3.2 Function Documentation

7.3.2.1 cmpEgz()

```
bool cmpEgz (  
    const Studentas & a,  
    const Studentas & b)
```

egzamino rezultatų palyginimas

Definition at line [364](#) of file [funk.cpp](#).

7.3.2.2 cmpMedrez()

```
bool cmpMedrez (  
    const Studentas & a,  
    const Studentas & b)
```

medianos rezultatų palyginimas

Definition at line [359](#) of file [funk.cpp](#).

7.3.2.3 cmpPavarde()

```
bool cmpPavarde (
    const Studentas & a,
    const Studentas & b)
```

pavardės palyginimas

Definition at line 349 of file [funk.cpp](#).

7.3.2.4 cmpRez()

```
bool cmpRez (
    const Studentas & a,
    const Studentas & b)
```

rezultatų palyginimas

Definition at line 354 of file [funk.cpp](#).

7.3.2.5 cmpVardas()

```
bool cmpVardas (
    const Studentas & a,
    const Studentas & b)
```

vardo palyginimas

Definition at line 344 of file [funk.cpp](#).

7.3.2.6 failgeneravimas()

```
void failgeneravimas ()
```

generuoja 5 failus su atsitiktiniais studentais

Definition at line 385 of file [funk.cpp](#).

7.3.2.7 failinputas()

```
void failinputas (
    Vector< Studentas > & grupe,
    string reading)
```

7.3.2.8 failoutputas()

```
void failoutputas (
    const Vector< Studentas > & grupe,
    string writing)
```

7.3.2.9 genirasymas()

```
void genirasymas (
    Vector< Studentas > & grupe,
    Vector< Studentas > & protingi,
    Vector< Studentas > & neprotingi,
    string writing_good,
    string writing_bad,
    int rik_pasirinkimas,
    SkirstymoStrategija strategija)
```

7.3.2.10 inputas()

```
void inputas (
    Vector< Studentas > & grupe)
```

7.3.2.11 logResults()

```
void logResults (
    const string & container,
    int size,
    double read_t,
    double sort_t,
    double split_t)
```

Definition at line 559 of file [funk.cpp](#).

7.3.2.12 lytgen()

```
std::string lytgen ()
```

random lyties generavimas

Definition at line 338 of file [funk.cpp](#).

7.3.2.13 matuoti_perskirstymus()

```
void matuoti_perskirstymus (
    int kiekis)
```

Matuoja perskirstymus std::vector ir [Vector](#).

Definition at line 658 of file [funk.cpp](#).

7.3.2.14 operator<<()

```
ostream & operator<< (
    ostream & os,
    const Studentas & s)
```

output operatorius klasei

Definition at line 602 of file [funk.cpp](#).

7.3.2.15 operator>>()

```
istream & operator>> (
    istream & is,
    Studentas & s)
```

input operatorius klasei

Definition at line 611 of file [funk.cpp](#).

7.3.2.16 outputas()

```
void outputas (
    const Vector< Studentas > & grupe)
```

7.3.2.17 randominputas()

```
void randominputas (
    Vector< Studentas > & grupe,
    bool ArGeneruotiVardus)
```

7.3.2.18 randompavarde()

```
string randompavarde (
    string lytis)
```

random pavardės generavimas iš listo

Definition at line 325 of file [funk.cpp](#).

7.3.2.19 randomvardas()

```
string randomvardas (
    string lytis)
```

random vardo generavimas iš listo

Definition at line 317 of file [funk.cpp](#).

7.3.2.20 rikiuoti()

```
void rikiuoti (
    Vector< Studentas > & grupe,
    int pasirinkimas)
```

7.3.2.21 skirstyti()

```
void skirstyti (
    const Vector< Studentas > & grupe,
    Vector< Studentas > & protingi,
    Vector< Studentas > & neprotingi)
```

7.3.2.22 skirstyti2()

```
void skirstyti2 (
    Vector< Studentas > & grupe,
    Vector< Studentas > & neprotingi)
```

7.3.2.23 skirstyti3()

```
void skirstyti3 (
    Vector< Studentas > & grupe,
    Vector< Studentas > & neprotingi)
```

7.3.2.24 uzdmat()

```
void uzdmat (
    int vektorodydis)
```

matuoja, kiek laiko uztrunka uzpildyti std::vector ir [Vector](#)

Definition at line [631](#) of file [funk.cpp](#).

7.3.3 Variable Documentation

7.3.3.1 fgalunes

```
const Vector<string> fgalunes = {"iene", "aite", "yte"}
```

Definition at line [23](#) of file [studentai.h](#).

7.3.3.2 fvardai

```
const Vector<string> fvardai = {"Egle", "Viktorija", "Vakare", "Inga", "Ema", "Marija", "Janina"}
```

Definition at line [20](#) of file [studentai.h](#).

7.3.3.3 irasymo_failas

```
const string irasymo_failas = "kursiokai.txt"
```

Definition at line [164](#) of file [studentai.h](#).

7.3.3.4 irasymo_failas_blogas

```
const string irasymo_failas_blogas = "kursiokai_blogi.txt"
```

Definition at line 166 of file [studentai.h](#).

7.3.3.5 irasymo_failas_geras

```
const string irasymo_failas_geras = "kursiokai_geri.txt"
```

Definition at line 165 of file [studentai.h](#).

7.3.3.6 mgalunes

```
const Vector<string> mgalunes = {"as", "aitis"}
```

Definition at line 22 of file [studentai.h](#).

7.3.3.7 mvardai

```
const Vector<string> mvardai = {"Vytenis", "Tomas", "Jonas", "Matas", "Simas", "Mantas",  
"Arnas"}
```

Definition at line 19 of file [studentai.h](#).

7.3.3.8 pavardes

```
const Vector<string> pavardes = {"Macerausk", "Jankausk", "Kazlausk", "Svilpausk", "Drugeliausk",  
"Briedausk"}
```

Definition at line 21 of file [studentai.h](#).

7.4 studentai.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STUDENTAI_H
00002 #define STUDENTAI_H
00003
00004 #pragma once
00005 #include <iostream>
00006 // #include <Vector>
00007 #include <string>
00008 #include <algorithm>
00009 #include "Vector2.h"
00010
00011 using namespace std;
00012
00013
00014 namespace my {
00015     template <typename T>
00016     using Vector = ::Vector<T>;
00017 }
00018
00019 const Vector<string> mvardai = {"Vytenis", "Tomas", "Jonas", "Matas", "Simas", "Mantas", "Arnas"};
00020 const Vector<string> fvardai = {"Egle", "Viktorija", "Vakare", "Inga", "Ema", "Marija", "Janina"};
```

```

00021 const Vector<string> pavardes = {"Macerausk", "Jankausk", "Kazlausk", "Svilpausk", "Drugeliausk",
    "Briedausk"};
00022 const Vector<string> mgalunes = {"as", "aitis"};
00023 const Vector<string> fgalunes = {"iene", "aite", "yte"};
00024
00025 enum class SkirstymoStrategija
00026 {
00027     PIRMAS,
00028     ANTRAS,
00029     TRECIAS
00030 };
00031
00032 class Zmogus {
00033 protected:
00034     string vardas;
00035     string pavarde;
00036
00037 public:
00038     Zmogus(const string& v = "A", const string& p = "BB")
00039     : vardas(v), pavarde(p)
00040     {}
00041     virtual ~Zmogus() {}
00042
00043     virtual void info() const = 0; // kad veiktu abstrakciai
00044
00045     string& refVardas() { return vardas; }
00046     string& refPavarde() { return pavarde; }
00047     string getVardas() const { return vardas; }
00048     string getPavarde() const { return pavarde; }
00049     void setVardas(const string& v) { vardas = v; }
00050     void setPavarde(const string& p) { pavarde = p; }
00051
00052     Zmogus(const Zmogus&) = default;
00053     Zmogus& operator=(const Zmogus&) = default;
00054     Zmogus(Zmogus&&) noexcept = default;
00055     Zmogus& operator=(Zmogus&&) noexcept = default;
00056 };
00057
00058 class Studentas : public Zmogus {
00059 private:
00060     Vector<int> paz;
00061     int egz = 0;
00062     double rez = 0.0;
00063     double medrez = 0.0;
00064
00065 public:
00066     Studentas(const string& v = "A",
00067               const string& p = "BB",
00068               const Vector<int>& paz_ = {},
00069               int egz_ = 0)
00070     : Zmogus(v, p), // zmogaus konstruktorius
00071       paz(paz_),
00072       egz(egz_),
00073       rez(0.0),
00074       medrez(0.0)
00075     {}
00076     vardas = v;
00077     pavarde = p;
00078
00079     if (!paz.empty())
00080         skaiciuoti();
00081
00082     ~Studentas() = default;
00083
00084 // Rule of Five
00085
00086 Studentas(const Studentas& other)
00087 : Zmogus(other),
00088   paz(other.paz),
00089   egz(other.egz),
00090   rez(other.rez),
00091   medrez(other.medrez)
00092 {}
00093
00094 Studentas& operator=(const Studentas& other) {
00095     if (this != &other) {
00096         Zmogus::operator=(other);
00097         paz = other.paz;
00098         egz = other.egz;
00099         rez = other.rez;
00100         medrez = other.medrez;
00101     }
00102     return *this;
00103 }
00104
00105 Studentas(Studentas&& other) noexcept
00106 : Zmogus(std::move(other)),

```

```

00116     paz(std::move(other.paz)),
00117     egz(other egz),
00118     rez(other.rez),
00119     medrez(other.medrez)
00120 {
00121     other.egz = 0;
00122     other.rez = 0;
00123     other.medrez = 0;
00124 }
00125
00127 Studentas& operator=(Studentas&& other) noexcept {
00128     if (this != &other) {
00129         Zmogus::operator=(std::move(other));
00130         paz = std::move(other.paz);
00131         egz = other.egz;
00132         rez = other.rez;
00133         medrez = other.medrez;
00134
00135         other.egz = 0;
00136         other.rez = 0;
00137         other.medrez = 0;
00138     }
00139     return *this;
00140 }
00141
00142 void info() const override { // kad veiktu ne abstrakciai?
00143     cout << "Studentas: " << vardas << " " << pavarde << endl;
00144 }
00145
00146 Vector<int>& refPaz() { return paz; }
00147 int& refEgz() { return egz; }
00148 double& refRez() { return rez; }
00149 double& refMedrez() { return medrez; }
00150 void clearPaz() { paz.clear(); }
00151 double getRez() const { return rez; }
00152 double getMedrez() const { return medrez; }
00153 int getEgz() const { return egz; }
00154
00155 const Vector<int>& getPaz() const { return paz; }
00156
00157 void addPaz(int p) { paz.push_back(p); }
00158 void setEgz(int e) { egz = e; }
00159
00160
00161 void skaiciuoti();
00162 };
00163
00164 const string irasyimo_failas = "kursiokai.txt";
00165 const string irasyimo_failas_geras = "kursiokai_geri.txt";
00166 const string irasyimo_failas_blogas = "kursiokai_blogi.txt";
00167
00168 void inputas(Vector<Studentas> &grupe);
00169 void failinputas(Vector<Studentas> &grupe, string reading);
00170 void randominputas(Vector<Studentas> &grupe, bool ArGeneruotiVardus);
00171 void outputas(const Vector<Studentas> &grupe);
00172 void failoutputas(const Vector<Studentas> &grupe, string writing);
00173 void failgeneravimas();
00174 void genirasyimas(Vector<Studentas> &grupe, Vector<Studentas> &protingi, Vector<Studentas> &neprotingi,
00175     string writing_good, string writing_bad, int rik_pasirinkimas, SkirstymoStrategija strategija);
00176 std::string lytgen();
00177 string randomvardas(string lytis);
00178 string randompavarde(string lytis);
00179 void rikiuoti(Vector<Studentas> &grupe, int pasirinkimas);
00180 bool cmpVardas(const Studentas &a, const Studentas &b);
00181 bool cmpPavarde(const Studentas &a, const Studentas &b);
00182 bool cmpRez(const Studentas &a, const Studentas &b);
00183 bool cmpMedrez(const Studentas &a, const Studentas &b);
00184 bool cmpEgz(const Studentas &a, const Studentas &b);
00185 void skirstyti(const Vector<Studentas> &grupe, Vector<Studentas> &protingi, Vector<Studentas> &
    neprotingi);
00186 void logResults(const string& container, int size,
00187     double read_t, double sort_t, double split_t);
00188 void skirstyti2(Vector<Studentas> &grupe, Vector<Studentas> &neprotingi);
00189 void skirstyti3(Vector<Studentas> &grupe, Vector<Studentas> &neprotingi);
00190
00191 void uzdmatis(int vektordydis);
00192 void matuoti_perskirstymus(int kiekis);
00193
00194 ostream& operator<<(ostream& os, const Studentas& s);
00195 istream& operator>>(istream& is, Studentas& s);
00196
00197
00198 #endif

```


7.5 testing.cpp File Reference

```
#include "catch.hpp"
#include "studentai.h"
#include <type_traits>
#include <vector>
#include <iostream>
#include <string>
#include <algorithm>
#include "Vector2.h"
```

Macros

- `#define CATCH_CONFIG_MAIN`

Functions

- `TEST_CASE` ("Studentu klases testas")
- `TEST_CASE` ("Zmogus yra abstract")
- `TEST_CASE` ("Ar studentas yra abstract?")
- `Studentas makeStudent` ()
- `TEST_CASE` ("Studentas copy konstruktorius")
- `TEST_CASE` ("Studentas copy assignment")
- `TEST_CASE` ("Studentas move konostruktorius")
- `TEST_CASE` ("Studentas move assignment")
- `TEST_CASE` ("Studentas move a -> a")
- `TEST_CASE` ("Studentas output operator >>")
- `TEST_CASE` ("Studentas input operator >>")
- `TEST_CASE` ("Vector default constructor")
- `TEST_CASE` ("Vector initializer_list constructor")
- `TEST_CASE` ("Vector fill constructor")
- `TEST_CASE` ("Vector copy constructor")
- `TEST_CASE` ("Vector copy assignment")
- `TEST_CASE` ("Vector move constructor")
- `TEST_CASE` ("Vector move assignment")
- `TEST_CASE` ("Vector push_back and pop_back")
- `TEST_CASE` ("Vector reserve increases capacity but not size")
- `TEST_CASE` ("Vector resize larger")
- `TEST_CASE` ("Vector resize smaller")
- `TEST_CASE` ("Vector at() throws on out of range")
- `TEST_CASE` ("Vector front() and back()")
- `TEST_CASE` ("Vector iterators work")
- `TEST_CASE` ("Vector reverse iterators")
- `TEST_CASE` ("Vector insert at middle")
- `TEST_CASE` ("Vector erase single element")
- `TEST_CASE` ("Vector erase range")
- `TEST_CASE` ("Vector comparison operators")
- `TEST_CASE` ("Vector swap")
- `TEST_CASE` ("Vector clear")
- `TEST_CASE` ("Vector shrink_to_fit reduces capacity to size")
- `TEST_CASE` ("Vector data() returns pointer to underlying array")
- `TEST_CASE` ("Vector cbegin() and cend() iterate correctly")
- `TEST_CASE` ("Vector const reverse iterators work")
- `TEST_CASE` ("Vector emplace_back constructs in-place")
- `TEST_CASE` ("Vector max_size is large enough")

7.5.1 Macro Definition Documentation

7.5.1.1 CATCH_CONFIG_MAIN

```
#define CATCH_CONFIG_MAIN
```

Definition at line 1 of file [testing.cpp](#).

7.5.2 Function Documentation

7.5.2.1 makeStudent()

```
Studentas makeStudent ()
```

Definition at line 37 of file [testing.cpp](#).

7.5.2.2 TEST_CASE() [1/37]

```
TEST_CASE (
    "Ar studentas yra abstract?" )
```

Definition at line 33 of file [testing.cpp](#).

7.5.2.3 TEST_CASE() [2/37]

```
TEST_CASE (
    "Studentas copy assignment" )
```

Definition at line 60 of file [testing.cpp](#).

7.5.2.4 TEST_CASE() [3/37]

```
TEST_CASE (
    "Studentas copy konstruktorius" )
```

Definition at line 48 of file [testing.cpp](#).

7.5.2.5 TEST_CASE() [4/37]

```
TEST_CASE (
    "Studentas input operator >>" )
```

Definition at line 153 of file [testing.cpp](#).

7.5.2.6 TEST_CASE() [5/37]

```
TEST_CASE (
    "Studentas move a -> a" )
```

Definition at line 126 of file [testing.cpp](#).

7.5.2.7 TEST_CASE() [6/37]

```
TEST_CASE (
    "Studentas move assignment" )
```

Definition at line 101 of file [testing.cpp](#).

7.5.2.8 TEST_CASE() [7/37]

```
TEST_CASE (
    "Studentas move konostruktorius" )
```

Definition at line 81 of file [testing.cpp](#).

7.5.2.9 TEST_CASE() [8/37]

```
TEST_CASE (
    "Studentas output operator >>" )
```

Definition at line 140 of file [testing.cpp](#).

7.5.2.10 TEST_CASE() [9/37]

```
TEST_CASE (
    "Studentu klases testas" )
```

Definition at line 11 of file [testing.cpp](#).

7.5.2.11 TEST_CASE() [10/37]

```
TEST_CASE (
    "Vector at() throws on out of range" )
```

Definition at line 269 of file [testing.cpp](#).

7.5.2.12 TEST_CASE() [11/37]

```
TEST_CASE (
    "Vector cbegin() and cend() iterate correctly" )
```

Definition at line 370 of file [testing.cpp](#).

7.5.2.13 TEST_CASE() [12/37]

```
TEST_CASE (
    "Vector clear" )
```

Definition at line [343](#) of file [testing.cpp](#).

7.5.2.14 TEST_CASE() [13/37]

```
TEST_CASE (
    "Vector comparison operators" )
```

Definition at line [320](#) of file [testing.cpp](#).

7.5.2.15 TEST_CASE() [14/37]

```
TEST_CASE (
    "Vector const reverse iterators work" )
```

Definition at line [380](#) of file [testing.cpp](#).

7.5.2.16 TEST_CASE() [15/37]

```
TEST_CASE (
    "Vector copy assignment" )
```

Definition at line [197](#) of file [testing.cpp](#).

7.5.2.17 TEST_CASE() [16/37]

```
TEST_CASE (
    "Vector copy constructor" )
```

Definition at line [186](#) of file [testing.cpp](#).

7.5.2.18 TEST_CASE() [17/37]

```
TEST_CASE (
    "Vector data() returns pointer to underlying array" )
```

Definition at line [360](#) of file [testing.cpp](#).

7.5.2.19 TEST_CASE() [18/37]

```
TEST_CASE (
    "Vector default constructor" )
```

Definition at line [165](#) of file [testing.cpp](#).

7.5.2.20 TEST_CASE() [19/37]

```
TEST_CASE (
    "Vector emplace_back constructs in-place" )
```

Definition at line 389 of file [testing.cpp](#).

7.5.2.21 TEST_CASE() [20/37]

```
TEST_CASE (
    "Vector erase range" )
```

Definition at line 311 of file [testing.cpp](#).

7.5.2.22 TEST_CASE() [21/37]

```
TEST_CASE (
    "Vector erase single element" )
```

Definition at line 303 of file [testing.cpp](#).

7.5.2.23 TEST_CASE() [22/37]

```
TEST_CASE (
    "Vector fill constructor" )
```

Definition at line 180 of file [testing.cpp](#).

7.5.2.24 TEST_CASE() [23/37]

```
TEST_CASE (
    "Vector front() and back()" )
```

Definition at line 274 of file [testing.cpp](#).

7.5.2.25 TEST_CASE() [24/37]

```
TEST_CASE (
    "Vector initializer_list constructor" )
```

Definition at line 172 of file [testing.cpp](#).

7.5.2.26 TEST_CASE() [25/37]

```
TEST_CASE (
    "Vector insert at middle" )
```

Definition at line 295 of file [testing.cpp](#).

7.5.2.27 TEST_CASE() [26/37]

```
TEST_CASE (
    "Vector iterators work" )
```

Definition at line 280 of file [testing.cpp](#).

7.5.2.28 TEST_CASE() [27/37]

```
TEST_CASE (
    "Vector max_size is large enough" )
```

Definition at line 403 of file [testing.cpp](#).

7.5.2.29 TEST_CASE() [28/37]

```
TEST_CASE (
    "Vector move assignment" )
```

Definition at line 219 of file [testing.cpp](#).

7.5.2.30 TEST_CASE() [29/37]

```
TEST_CASE (
    "Vector move constructor" )
```

Definition at line 209 of file [testing.cpp](#).

7.5.2.31 TEST_CASE() [30/37]

```
TEST_CASE (
    "Vector push_back and pop_back" )
```

Definition at line 230 of file [testing.cpp](#).

7.5.2.32 TEST_CASE() [31/37]

```
TEST_CASE (
    "Vector reserve increases capacity but not size" )
```

Definition at line 244 of file [testing.cpp](#).

7.5.2.33 TEST_CASE() [32/37]

```
TEST_CASE (
    "Vector resize larger" )
```

Definition at line 251 of file [testing.cpp](#).

7.5.2.34 TEST_CASE() [33/37]

```
TEST_CASE (
    "Vector resize smaller" )
```

Definition at line 260 of file [testing.cpp](#).

7.5.2.35 TEST_CASE() [34/37]

```
TEST_CASE (
    "Vector reverse iterators" )
```

Definition at line 289 of file [testing.cpp](#).

7.5.2.36 TEST_CASE() [35/37]

```
TEST_CASE (
    "Vector shrink_to_fit reduces capacity to size" )
```

Definition at line 351 of file [testing.cpp](#).

7.5.2.37 TEST_CASE() [36/37]

```
TEST_CASE (
    "Vector swap" )
```

Definition at line 331 of file [testing.cpp](#).

7.5.2.38 TEST_CASE() [37/37]

```
TEST_CASE (
    "Zmogus yra abstract" )
```

Definition at line 29 of file [testing.cpp](#).

7.6 testing.cpp

[Go to the documentation of this file.](#)

```

00001 #define CATCH_CONFIG_MAIN
00002 #include "catch.hpp"
00003 #include "studentai.h"
00004 #include <type_traits>
00005 #include <vector>
00006 #include <iostream>
00007 #include <string>
00008 #include <algorithm>
00009 #include "Vector2.h"
00010
00011 TEST_CASE("Studentu klases testas") {
00012     Studentas A;
00013
00014     A.setVardas("Zygimantas");
00015     A.setPavarde("Rauba");
00016     A.addPaz(10);
00017     A.setEgz(6);
00018
00019     A.skaiciuoti();
00020
00021     REQUIRE(A.getVardas() == "Zygimantas");
00022     REQUIRE(A.getPavarde() == "Rauba");
00023     REQUIRE(A.getPaz() == my::Vector<int>{10});
00024     REQUIRE(A.getEgz() == 6);
00025     REQUIRE(A.getRez() == 7.60);
00026     REQUIRE(A.getMedrez() == 7.60);
00027 }
00028
00029 TEST_CASE("Zmogus yra abstract") {
00030     REQUIRE(std::is_abstract_v<Zmogus>);
00031 }
00032
00033 TEST_CASE("Ar studentas yra abstract?") {
00034     REQUIRE_FALSE(std::is_abstract_v<Studentas>);
00035 }
00036
00037 Studentas makeStudent() {
00038     Studentas s;
00039     s.setVardas("Jonas");
00040     s.setPavarde("Jonaitis");
00041     s.addPaz(10);
00042     s.addPaz(9);
00043     s.setEgz(8);
00044     s.skaiciuoti();
00045     return s;
00046 }
00047
00048 TEST_CASE("Studentas copy konstruktorius") {
00049     Studentas a = makeStudent();
00050     Studentas b(a);
00051
00052     REQUIRE(b.getVardas() == "Jonas");
00053     REQUIRE(b.getPavarde() == "Jonaitis");
00054     REQUIRE(b.getPaz() == my::Vector<int>{10, 9});
00055     REQUIRE(b.getEgz() == 8);
00056     REQUIRE(b.getRez() == 8.60);
00057     REQUIRE(b.getMedrez() == 8.60);
00058 }
00059
00060 TEST_CASE("Studentas copy assignment") {
00061     Studentas a = makeStudent();
00062     Studentas b;
00063     b = a;
00064
00065     REQUIRE(b.getVardas() == "Jonas");
00066     REQUIRE(b.getPavarde() == "Jonaitis");
00067     REQUIRE(b.getPaz() == my::Vector<int>{10, 9});
00068     REQUIRE(b.getEgz() == 8);
00069     REQUIRE(b.getRez() == 8.60);
00070     REQUIRE(b.getMedrez() == 8.60);
00071
00072     b.addPaz(5);
00073     a.addPaz(2);
00074     a.skaiciuoti();
00075     b.skaiciuoti();
00076
00077     REQUIRE(b.getRez() == 8.00);
00078     REQUIRE(a.getRez() == 7.60);
00079 }
00080
00081 TEST_CASE("Studentas move konstruktorius") {
00082     Studentas a = makeStudent();

```



```

00083     Studentas b(move(a));
00084
00085     REQUIRE(b.getVardas() == "Jonas");
00086     REQUIRE(b.getPavarde() == "Jonaitis");
00087     REQUIRE(b.getPaz() == my::Vector<int>{10, 9});
00088     REQUIRE(b.getEgz() == 8);
00089     REQUIRE(b.getRez() == 8.60);
00090     REQUIRE(b.getMedrez() == 8.60);
00091
00092     b.addPaz(5);
00093     a.addPaz(2);
00094     a.skaiciuoti();
00095     b.skaiciuoti();
00096
00097     REQUIRE(b.getRez() == 8.00);
00098     REQUIRE(a.getRez() == 0.80); // del to, kad a tampa tuscias ir vienintelis pazymys yra 2
00099 }
00100
00101 TEST_CASE("Studentas move assignment") {
00102     Studentas a = makeStudent();
00103     Studentas b;
00104     b = move(a);
00105
00106
00107     REQUIRE(b.getVardas() == "Jonas");
00108     REQUIRE(b.getPavarde() == "Jonaitis");
00109     REQUIRE(b.getPaz() == my::Vector<int>{10, 9});
00110     REQUIRE(b.getEgz() == 8);
00111     REQUIRE(b.getRez() == 8.60);
00112     REQUIRE(b.getMedrez() == 8.60);
00113
00114     b.addPaz(5);
00115     a.addPaz(2);
00116     a.skaiciuoti();
00117     b.skaiciuoti();
00118
00119     REQUIRE(b.getRez() == 8.00);
00120     REQUIRE(a.getRez() == 0.80); // del to, kad a tampa tuscias ir vienintelis pazymys yra 2
00121     REQUIRE(a.getVardas().empty());
00122     REQUIRE(a.getPavarde().empty());
00123     REQUIRE(a.getEgz() == 0);
00124 }
00125
00126 TEST_CASE("Studentas move a -> a") {
00127     Studentas a = makeStudent();
00128     a = move(a);
00129
00130     REQUIRE(a.getVardas() == "Jonas");
00131     REQUIRE(a.getPavarde() == "Jonaitis");
00132     REQUIRE(a.getPaz() == my::Vector<int>{10, 9});
00133     REQUIRE(a.getEgz() == 8);
00134     REQUIRE(a.getRez() == 8.60);
00135     REQUIRE(a.getMedrez() == 8.60);
00136
00137     REQUIRE(a.getRez() == 8.60); // del to, kad a tampa tuscias ir vienintelis pazymys yra 2
00138 }
00139
00140 TEST_CASE("Studentas output operator »") {
00141     Studentas s = makeStudent();
00142
00143     std::ostringstream out;
00144     out << s;
00145
00146     std::string result = out.str();
00147
00148     REQUIRE(result.find("Jonas") != std::string::npos);
00149     REQUIRE(result.find("Jonaitis") != std::string::npos);
00150     REQUIRE(result.find("8.60") != std::string::npos);
00151 }
00152
00153 TEST_CASE("Studentas input operator »") {
00154     std::istringstream input("Jonas Jonaitis 10 9 8");
00155
00156     Studentas s;
00157     input >> s;
00158
00159     REQUIRE(s.getVardas() == "Jonas");
00160     REQUIRE(s.getPavarde() == "Jonaitis");
00161     REQUIRE(s.getPaz() == my::Vector<int>{10, 9});
00162     REQUIRE(s.getEgz() == 8);
00163 }
00164
00165 TEST_CASE("Vector default constructor") {
00166     Vector<int> v;
00167     REQUIRE(v.size() == 0);
00168     REQUIRE(v.capacity() == 0);
00169     REQUIRE(v.empty());

```

```

00170 }
00171
00172 TEST_CASE("Vector initializer_list constructor") {
00173     Vector<int> v{1, 2, 3};
00174     REQUIRE(v.size() == 3);
00175     REQUIRE(v[0] == 1);
00176     REQUIRE(v[1] == 2);
00177     REQUIRE(v[2] == 3);
00178 }
00179
00180 TEST_CASE("Vector fill constructor") {
00181     Vector<int> v(5, 7);
00182     REQUIRE(v.size() == 5);
00183     for (int x : v) REQUIRE(x == 7);
00184 }
00185
00186 TEST_CASE("Vector copy constructor") {
00187     Vector<int> a{1, 2, 3};
00188     Vector<int> b(a);
00189
00190     REQUIRE(b.size() == 3);
00191     REQUIRE(b == a);
00192
00193     b[0] = 99;
00194     REQUIRE(a[0] == 1); // deep copy
00195 }
00196
00197 TEST_CASE("Vector copy assignment") {
00198     Vector<int> a{1, 2, 3};
00199     Vector<int> b;
00200     b = a;
00201
00202     REQUIRE(b.size() == 3);
00203     REQUIRE(b == a);
00204
00205     b[1] = 77;
00206     REQUIRE(a[1] == 2);
00207 }
00208
00209 TEST_CASE("Vector move constructor") {
00210     Vector<int> a{1, 2, 3};
00211     Vector<int> b(std::move(a));
00212
00213     REQUIRE(b.size() == 3);
00214     REQUIRE(b[0] == 1);
00215     REQUIRE(a.size() == 0);
00216     REQUIRE(a.capacity() == 0);
00217 }
00218
00219 TEST_CASE("Vector move assignment") {
00220     Vector<int> a{1, 2, 3};
00221     Vector<int> b;
00222     b = std::move(a);
00223
00224     REQUIRE(b.size() == 3);
00225     REQUIRE(b[2] == 3);
00226     REQUIRE(a.size() == 0);
00227     REQUIRE(a.capacity() == 0);
00228 }
00229
00230 TEST_CASE("Vector push_back and pop_back") {
00231     Vector<int> v;
00232     v.push_back(10);
00233     v.push_back(20);
00234     v.push_back(30);
00235
00236     REQUIRE(v.size() == 3);
00237     REQUIRE(v.back() == 30);
00238
00239     v.pop_back();
00240     REQUIRE(v.size() == 2);
00241     REQUIRE(v.back() == 20);
00242 }
00243
00244 TEST_CASE("Vector reserve increases capacity but not size") {
00245     Vector<int> v;
00246     v.reserve(50);
00247     REQUIRE(v.capacity() >= 50);
00248     REQUIRE(v.size() == 0);
00249 }
00250
00251 TEST_CASE("Vector resize larger") {
00252     Vector<int> v{1, 2, 3};
00253     v.resize(5, 9);
00254
00255     REQUIRE(v.size() == 5);
00256     REQUIRE(v[3] == 9);

```

```

00257     REQUIRE(v[4] == 9);
00258 }
00259
00260 TEST_CASE("Vector resize smaller") {
00261     Vector<int> v{1, 2, 3, 4, 5};
00262     v.resize(2);
00263
00264     REQUIRE(v.size() == 2);
00265     REQUIRE(v[0] == 1);
00266     REQUIRE(v[1] == 2);
00267 }
00268
00269 TEST_CASE("Vector at() throws on out of range") {
00270     Vector<int> v{1, 2, 3};
00271     REQUIRE_THROWS_AS(v.at(5), std::out_of_range);
00272 }
00273
00274 TEST_CASE("Vector front() and back()") {
00275     Vector<int> v{10, 20, 30};
00276     REQUIRE(v.front() == 10);
00277     REQUIRE(v.back() == 30);
00278 }
00279
00280 TEST_CASE("Vector iterators work") {
00281     Vector<int> v{1, 2, 3};
00282     int sum = 0;
00283     for (auto it = v.begin(); it != v.end(); ++it)
00284         sum += *it;
00285
00286     REQUIRE(sum == 6);
00287 }
00288
00289 TEST_CASE("Vector reverse iterators") {
00290     Vector<int> v{1, 2, 3};
00291     REQUIRE(*v.rbegin() == 3);
00292     REQUIRE(*(v.rbegin() + 1) == 2);
00293 }
00294
00295 TEST_CASE("Vector insert at middle") {
00296     Vector<int> v{1, 2, 4};
00297     v.insert(v.begin() + 2, 3);
00298
00299     REQUIRE(v.size() == 4);
00300     REQUIRE(v[2] == 3);
00301 }
00302
00303 TEST_CASE("Vector erase single element") {
00304     Vector<int> v{1, 2, 3, 4};
00305     v.erase(v.begin() + 1);
00306
00307     REQUIRE(v.size() == 3);
00308     REQUIRE(v[1] == 3);
00309 }
00310
00311 TEST_CASE("Vector erase range") {
00312     Vector<int> v{1, 2, 3, 4, 5};
00313     v.erase(v.begin() + 1, v.begin() + 4);
00314
00315     REQUIRE(v.size() == 2);
00316     REQUIRE(v[0] == 1);
00317     REQUIRE(v[1] == 5);
00318 }
00319
00320 TEST_CASE("Vector comparison operators") {
00321     Vector<int> a{1, 2, 3};
00322     Vector<int> b{1, 2, 3};
00323     Vector<int> c{1, 2, 4};
00324
00325     REQUIRE(a == b);
00326     REQUIRE(a != c);
00327     REQUIRE(a < c);
00328     REQUIRE(c > b);
00329 }
00330
00331 TEST_CASE("Vector swap") {
00332     Vector<int> a{1, 2};
00333     Vector<int> b{9, 8, 7};
00334
00335     a.swap(b);
00336
00337     REQUIRE(a.size() == 3);
00338     REQUIRE(b.size() == 2);
00339     REQUIRE(a[0] == 9);
00340     REQUIRE(b[1] == 2);
00341 }
00342
00343 TEST_CASE("Vector clear") {

```

```

00344     Vector<int> v{1, 2, 3};
00345     v.clear();
00346
00347     REQUIRE(v.size() == 0);
00348     REQUIRE(v.capacity() >= 3); // capacity stays
00349 }
00350
00351 TEST_CASE("Vector shrink_to_fit reduces capacity to size") {
00352     Vector<int> v{1, 2, 3};
00353     v.reserve(100);
00354     REQUIRE(v.capacity() >= 100);
00355
00356     v.shrink_to_fit();
00357     REQUIRE(v.capacity() == v.size());
00358 }
00359
00360 TEST_CASE("Vector data() returns pointer to underlying array") {
00361     Vector<int> v{10, 20, 30};
00362     int* ptr = v.data();
00363
00364     REQUIRE(ptr != nullptr);
00365     REQUIRE(ptr[0] == 10);
00366     REQUIRE(ptr[1] == 20);
00367     REQUIRE(ptr[2] == 30);
00368 }
00369
00370 TEST_CASE("Vector cbegin() and cend() iterate correctly") {
00371     Vector<int> v{1, 2, 3};
00372     int sum = 0;
00373
00374     for (auto it = v.cbegin(); it != v.cend(); ++it)
00375         sum += *it;
00376
00377     REQUIRE(sum == 6);
00378 }
00379
00380 TEST_CASE("Vector const reverse iterators work") {
00381     const Vector<int> v{1, 2, 3};
00382
00383     auto it = v.rbegin();
00384     REQUIRE(*it == 3);
00385     ++it;
00386     REQUIRE(*it == 2);
00387 }
00388
00389 TEST_CASE("Vector emplace_back constructs in-place") {
00390     struct TestObj {
00391         int x, y;
00392         TestObj(int a, int b) : x(a), y(b) {}
00393     };
00394
00395     Vector<TestObj> v;
00396     v.emplace_back(5, 7);
00397
00398     REQUIRE(v.size() == 1);
00399     REQUIRE(v[0].x == 5);
00400     REQUIRE(v[0].y == 7);
00401 }
00402
00403 TEST_CASE("Vector max_size is large enough") {
00404     Vector<int> v;
00405     REQUIRE(v.max_size() > 1000000); // sanity check
00406 }
00407

```

7.7 Vector2.h File Reference

```

#include <memory>
#include <stdexcept>
#include <initializer_list>
#include <algorithm>
#include <iterator>
#include <utility>
#include <limits>
#include <string>

```

Classes

- class `Vector< T, Allocator >`

7.8 Vector2.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include <memory>
00004 #include <stdexcept>
00005 #include <initializer_list>
00006 #include <algorithm>
00007 #include <iterator>
00008 #include <utility>
00009 #include <limits>
00010 #include <string>
00011
00012 template<typename T, typename Allocator = std::allocator<T>
00013 class Vector
00014 {
00015 public:
00016     using value_type          = T;
00017     using allocator_type      = Allocator;
00018     using size_type           = std::size_t;
00019     using difference_type     = std::ptrdiff_t;
00020     using reference           = T&;
00021     using const_reference     = const T&;
00022     using pointer             = T*;
00023     using const_pointer       = const T*;
00024
00025     using iterator            = T*;
00026     using const_iterator      = const T*;
00027     using reverse_iterator    = std::reverse_iterator<iterator>;
00028     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00029
00030 private:
00031     allocator_type _alloc;
00032     pointer _data;
00033     size_type _size;
00034     size_type _capacity;
00035
00036 private:
00037     void destroy_elements()
00038     {
00039         for (size_type i = 0; i < _size; ++i)
00040             std::allocator_traits<Allocator>::destroy(_alloc, _data + i);
00041
00042         _size = 0;
00043     }
00044
00045     void deallocate_storage()
00046     {
00047         if (_data)
00048         {
00049             std::allocator_traits<Allocator>::deallocate(
00050                 _alloc,
00051                 _data,
00052                 _capacity
00053             );
00054         }
00055         _data = nullptr;
00056         _capacity = 0;
00057     }
00058
00059     void reallocate(size_type new_capacity)
00060     {
00061         pointer new_data =
00062             std::allocator_traits<Allocator>::allocate(_alloc, new_capacity);
00063
00064         size_type constructed = 0;
00065
00066         try
00067         {
00068             for (; constructed < _size; ++constructed)
00069             {
00070                 std::allocator_traits<Allocator>::construct(
00071                     _alloc,
00072 
```

```

00074         new_data + constructed,
00075         std::move_if_noexcept(_data[constructed])
00076     );
00077 }
00078 }
00079 catch (...)
00080 {
00081     for (size_type i = 0; i < constructed; ++i)
00082     {
00083         std::allocator_traits<Allocator>::destroy(
00084             _alloc,
00085             new_data + i
00086         );
00087     }
00088
00089     std::allocator_traits<Allocator>::deallocate(
00090         _alloc,
00091         new_data,
00092         new_capacity
00093     );
00094
00095     throw;
00096 }
00097
00098 destroy_elements();
00099 deallocate_storage();
00100
00101 _data = new_data;
00102 _capacity = new_capacity;
00103 _size = constructed;
00104 }
00105
00106 void grow_if_needed()
00107 {
00108     if (_size >= _capacity)
00109     {
00110         size_type new_capacity = (_capacity == 0) ? 1 : (_capacity * 2);
00111         reallocate(new_capacity);
00112     }
00113 }
00114
00115 public:
00116
00117 /* =====
00118                      Constructors
00119 ===== */
00120
00121 Vector()
00122     : _data(nullptr),
00123       _size(0),
00124       _capacity(0)
00125 {
00126 }
00127
00128 explicit Vector(size_type count)
00129     : Vector()
00130 {
00131     resize(count);
00132 }
00133
00134 Vector(size_type count, const T& value)
00135     : Vector()
00136 {
00137     reserve(count);
00138
00139     for (size_type i = 0; i < count; ++i)
00140         push_back(value);
00141 }
00142
00143 Vector(std::initializer_list<T> init)
00144     : Vector()
00145 {
00146     reserve(init.size());
00147
00148     for (const auto& value : init)
00149         push_back(value);
00150 }
00151
00152 Vector(const Vector& other)
00153     : Vector()
00154 {
00155     reserve(other._size);
00156
00157     for (const auto& value : other)
00158         push_back(value);
00159 }
00160

```

```

00161     Vector(Vector&& other) noexcept
00162     : _alloc(std::move(other._alloc)),
00163       _data(other._data),
00164       _size(other._size),
00165       _capacity(other._capacity)
00166     {
00167         other._data = nullptr;
00168         other._size = 0;
00169         other._capacity = 0;
00170     }
00171
00172     ~Vector()
00173     {
00174         destroy_elements();
00175         deallocate_storage();
00176     }
00177
00178     /* =====
00179                                     Assignment
00180     ===== */
00181
00182     Vector& operator=(const Vector& other)
00183     {
00184         if (this == &other)
00185             return *this;
00186
00187         Vector temp(other);
00188         swap(temp);
00189
00190         return *this;
00191     }
00192
00193     Vector& operator=(Vector&& other) noexcept
00194     {
00195         if (this == &other)
00196             return *this;
00197
00198         destroy_elements();
00199         deallocate_storage();
00200
00201         _alloc = std::move(other._alloc);
00202         _data = other._data;
00203         _size = other._size;
00204         _capacity = other._capacity;
00205
00206         other._data = nullptr;
00207         other._size = 0;
00208         other._capacity = 0;
00209
00210         return *this;
00211     }
00212
00213     /* =====
00214                                     Iterators
00215     ===== */
00216
00217     iterator begin() noexcept { return _data; }
00218     const_iterator begin() const noexcept { return _data; }
00219
00220     const_iterator cbegin() const noexcept { return _data; }
00221
00222     iterator end() noexcept { return _data + _size; }
00223     const_iterator end() const noexcept { return _data + _size; }
00224
00225     const_iterator cend() const noexcept { return _data + _size; }
00226
00227     reverse_iterator rbegin() noexcept
00228     {
00229         return reverse_iterator(end());
00230     }
00231
00232     reverse_iterator rend() noexcept
00233     {
00234         return reverse_iterator(begin());
00235     }
00236
00237     const_reverse_iterator rbegin() const noexcept
00238     {
00239         return const_reverse_iterator(end());
00240     }
00241
00242     const_reverse_iterator rend() const noexcept
00243     {
00244         return const_reverse_iterator(begin());
00245     }
00246
00247     /* =====

```

```

00248                                     Capacity
00249     ===== */
00250
00251     bool empty() const noexcept
00252     {
00253         return _size == 0;
00254     }
00255
00256     size_type size() const noexcept
00257     {
00258         return _size;
00259     }
00260
00261     size_type capacity() const noexcept
00262     {
00263         return _capacity;
00264     }
00265
00266     size_type max_size() const noexcept
00267     {
00268         return std::numeric_limits<size_type>::max() / sizeof(T);
00269     }
00270
00271     void reserve(size_type new_capacity)
00272     {
00273         if (new_capacity > _capacity)
00274             reallocate(new_capacity);
00275     }
00276
00277     void shrink_to_fit()
00278     {
00279         if (_size < _capacity)
00280             reallocate(_size);
00281     }
00282
00283     void resize(size_type count)
00284     {
00285         resize(count, T());
00286     }
00287
00288     void resize(size_type count, const T& value)
00289     {
00290         if (count < _size)
00291         {
00292             for (size_type i = count; i < _size; ++i)
00293             {
00294                 std::allocator_traits<Allocator>::destroy(
00295                     _alloc,
00296                     _data + i
00297                 );
00298             }
00299             _size = count;
00300         }
00301         else if (count > _size)
00302         {
00303             reserve(count);
00304             while (_size < count)
00305                 push_back(value);
00306         }
00307     }
00308
00309
00310
00311     /* =====
00312                                     Element Access
00313     ===== */
00314
00315     reference operator[](size_type index)
00316     {
00317         return _data[index];
00318     }
00319
00320     const_reference operator[](size_type index) const
00321     {
00322         return _data[index];
00323     }
00324
00325     reference at(size_type index)
00326     {
00327         if (index >= _size)
00328             throw std::out_of_range("Vector::at");
00329         return _data[index];
00330     }
00331
00332
00333     const_reference at(size_type index) const
00334     {

```



```

00335         if (index >= _size)
00336             throw std::out_of_range("Vector::at");
00337
00338         return _data[index];
00339     }
00340
00341     reference front()
00342     {
00343         return _data[0];
00344     }
00345
00346     const_reference front() const
00347     {
00348         return _data[0];
00349     }
00350
00351     reference back()
00352     {
00353         return _data[_size - 1];
00354     }
00355
00356     const_reference back() const
00357     {
00358         return _data[_size - 1];
00359     }
00360
00361     pointer data() noexcept
00362     {
00363         return _data;
00364     }
00365
00366     const_pointer data() const noexcept
00367     {
00368         return _data;
00369     }
00370
00371     /* =====
00372        Modifiers
00373     ===== */
00374
00375     void clear() noexcept
00376     {
00377         destroy_elements();
00378     }
00379
00380     void push_back(const T& value)
00381     {
00382         grow_if_needed();
00383
00384         std::allocator_traits<Allocator>::construct(
00385             _alloc,
00386             _data + _size,
00387             value
00388         );
00389
00390         ++_size;
00391     }
00392
00393     void push_back(T&& value)
00394     {
00395         grow_if_needed();
00396
00397         std::allocator_traits<Allocator>::construct(
00398             _alloc,
00399             _data + _size,
00400             std::move(value)
00401         );
00402
00403         ++_size;
00404     }
00405
00406     template<typename... Args>
00407     reference emplace_back(Args&&... args)
00408     {
00409         grow_if_needed();
00410
00411         std::allocator_traits<Allocator>::construct(
00412             _alloc,
00413             _data + _size,
00414             std::forward<Args>(args)...)
00415         );
00416
00417         ++_size;
00418
00419         return back();
00420     }
00421

```

```

00422 void pop_back()
00423 {
00424     if (_size == 0)
00425         return;
00426
00427     --_size;
00428
00429     std::allocator_traits<Allocator>::destroy(
00430         _alloc,
00431         _data + _size
00432     );
00433 }
00434
00435 iterator insert(iterator pos, const T& value)
00436 {
00437     size_type index = pos - begin();
00438
00439     grow_if_needed();
00440
00441     pos = begin() + index;
00442
00443     for (iterator it = end(); it != pos; --it)
00444     {
00445         std::allocator_traits<Allocator>::construct(
00446             _alloc,
00447             it,
00448             std::move(*(it - 1))
00449         );
00450
00451         std::allocator_traits<Allocator>::destroy(
00452             _alloc,
00453             it - 1
00454         );
00455     }
00456
00457     std::allocator_traits<Allocator>::construct(
00458         _alloc,
00459         pos,
00460         value
00461     );
00462
00463     ++_size;
00464
00465     return pos;
00466 }
00467
00468 iterator erase(iterator pos)
00469 {
00470     size_type index = pos - begin();
00471
00472     // shift elements left with destroy+construct to keep destructors correct
00473     for (size_type i = index; i < _size - 1; ++i)
00474     {
00475         std::allocator_traits<Allocator>::destroy(_alloc, _data + i);
00476         std::allocator_traits<Allocator>::construct(
00477             _alloc,
00478             _data + i,
00479             std::move(_data[i + 1])
00480         );
00481     }
00482
00483     pop_back(); // destroys last element and decrements _size
00484     return begin() + index;
00485 }
00486 void swap(Vector& other) noexcept
00487 {
00488     std::swap(_alloc, other._alloc);
00489     std::swap(_data, other._data);
00490     std::swap(_size, other._size);
00491     std::swap(_capacity, other._capacity);
00492 }
00493
00494 /* =====
00495      Comparison
00496      ===== */
00497
00498 bool operator==(const Vector& other) const
00499 {
00500     if (_size != other._size)
00501         return false;
00502
00503     for (size_type i = 0; i < _size; ++i)
00504     {
00505         if (!(_data[i] == other._data[i]))
00506             return false;
00507     }
00508 }

```

```

00509         return true;
00510     }
00511
00512     bool operator!=(const Vector& other) const
00513     {
00514         return !(*this == other);
00515     }
00516
00517     bool operator<(const Vector& other) const
00518     {
00519         return std::lexicographical_compare(
00520             begin(),
00521             end(),
00522             other.begin(),
00523             other.end()
00524         );
00525     }
00526
00527     bool operator>(const Vector& other) const
00528     {
00529         return other < *this;
00530     }
00531
00532     bool operator<=(const Vector& other) const
00533     {
00534         return !(other < *this);
00535     }
00536
00537     bool operator>=(const Vector& other) const
00538     {
00539         return !(*this < other);
00540     }
00541
00542     iterator erase(iterator first, iterator last)
00543     {
00544         if (first == last) return first;
00545
00546         size_type index = first - begin();
00547         size_type count = last - first;
00548
00549         // shift tail left
00550         for (size_type i = index; i + count < _size; ++i)
00551         {
00552             std::allocator_traits<Allocator>::destroy(_alloc, _data + i);
00553             std::allocator_traits<Allocator>::construct(
00554                 _alloc,
00555                 _data + i,
00556                 std::move(_data[i + count])
00557             );
00558         }
00559
00560         // destroy trailing elements
00561         for (size_type i = 0; i < count; ++i)
00562             pop_back();
00563
00564         return begin() + index;
00565     }
00566 };

```

7.9 versija_v3.0.cpp File Reference

```

#include <iostream>
#include <iomanip>
#include <string>
#include <cmath>
#include <algorithm>
#include <random>
#include <ctime>
#include <fstream>
#include <sstream>
#include <chrono>
#include "studentai.h"
#include <stdexcept>
#include "Vector2.h"

```

Functions

- `int main ()`

7.9.1 Function Documentation

7.9.1.1 main()

```
int main ()
```

Definition at line 18 of file [versija_v3.0.cpp](#).

7.10 versija_v3.0.cpp

[Go to the documentation of this file.](#)

```
00001 #include <iostream>
00002 // #include <Vector>
00003 #include <iomanip>
00004 #include <string>
00005 #include <cmath>
00006 #include <algorithm>
00007 #include <random>
00008 #include <ctime>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <chrono>
00012 #include "studentai.h"
00013 #include <stdexcept>
00014 #include "Vector2.h"
00015
00016 using namespace std;
00017
00018 int main()
00019 {
00020     srand(time(NULL));
00021
00022     Vector<Studentas> grupe;
00023     Vector<Studentas> protingi;
00024     Vector<Studentas> neprotingi;
00025
00026     grupe.reserve(10000000);
00027
00028     /*
00029     cout << "VECTOR PILDYMU MATAVIMAS (2): " << endl << endl << endl;
00030     for (int i = 4; i <= 8; i++)
00031     {
00032         uzdmata(pow(10, i));
00033     }
00034     cout << "PERSKIRSTYMU MATAVIMAS (3): " << endl << endl << endl;
00035     matuoti_perskirstymus(100000000);
00036
00037     string genpas;
00038     cout << "Norite generuoti 5 failus? y/n " << endl;
00039     cin >> genpas;
00040     if (genpas == "y") failgeneravimas();
00041     */
00042
00043     int pasirinkimas = 4;
00044     string isvedimotipas, skaitymo_pasirinkimas, skaitymo_failas;
00045     cout << "Kokios norite ivesties? (1 - ranka arba is failo, 2 - generuoti tik pazymius, " << endl
00046     << " 3 - generuoti studentu vardus, pavardes ir pazymius, 4 - skaityti sugeneruotus failus; 5 arba
00047     kitas simbolis - baigti darba)" << endl;
00048     cin >> pasirinkimas;
00049
00050     int rikiavimo_pasirinkimas;
00051     cout << "Pagal ka rikiuoti?" << endl
00052     << "1 - vardas" << endl
00053     << "2 - pavarde" << endl
00054     << "3 - galutinis (vidurkis)" << endl
00055     << "4 - galutinis (mediana)" << endl
00056     << "5 - egzaminas" << endl;
```

```

00057     cin » rikiavimo_pasirinkimas;
00058
00059
00060     auto input_start = chrono::high_resolution_clock::now();
00061
00062     if (pasirinkimas == 1)
00063     {
00064         cout << "Ar skaityti duomenis is failo? (y - is failo / n - ranka)";
00065         cin » skaitymo_pasirinkimas;
00066         if (skaitymo_pasirinkimas == "y")
00067         {
00068             string skaitymo_failas;
00069             cout << "Iveskite failo pavadinima: ";
00070             cin » skaitymo_failas;
00071             failinputas(grupe, skaitymo_failas);
00072         }
00073         else inputas(grupe);
00074     }
00075     else if (pasirinkimas == 2)
00076     {
00077         randominputas(grupe, false);
00078     }
00079     else if (pasirinkimas == 3)
00080     {
00081         randominputas(grupe, true);
00082     }
00083     else if (pasirinkimas == 4)
00084     {
00085         genirasymas(grupe, protingi, neprotingi, irasymo_failas_geras, irasymo_failas_blogas,
rikiavimo_pasirinkimas, SkirstymoStrategija::TRECIAS);
00086         return 0;
00087     }
00088     else
00089     {
00090         cout << "darbas baigtas" << endl;
00091         return 0;
00092     }
00093
00094     auto input_end = chrono::high_resolution_clock::now();
00095     chrono::duration<double> input_diff = input_end - input_start;
00096     cout << "Duomeniu nuskaitymas uztruko: " << input_diff.count() << " s \n";
00097
00098     cout << "Ar rasyti i faila? (y/n)" << endl;
00099     cin » isvedimotipas;
00100
00101     auto start = std::chrono::high_resolution_clock::now(); // Paleisti
00102
00103     rikiuoti(grupe, rikiavimo_pasirinkimas);
00104     //skirstyti(grupe, protingi, neprotingi)
00105     //skirstyti2(grupe, neprotingi);
00106     skirstyti3(grupe, neprotingi);
00107
00108     if (isvedimotipas == "y")
00109     {
00110         cout << "pradedu rasyt" << endl;
00111         failoutputas(protingi, "kursiokai_geri.txt");
00112         failoutputas(grupe, "kursiokai_geri_grupe.txt");
00113         failoutputas(neprotingi, "kursiokai_blogi.txt");
00114         cout << "Baigiu rasyt" << endl;
00115     }
00116     else
00117     {
00118         outputas(grupe);
00119     }
00120
00121     // is pavyzdzio
00122     auto end = chrono::high_resolution_clock::now(); // Stabdyti
00123     chrono::duration<double> diff = end-start; // Skirtumas (s)
00124     cout << "Programos rikiavimas ir isvedimas uztruko: " << diff.count() << " s\n";
00125
00126     cout << "program finished." << endl;
00127     return 0;
00128 }

```


Index

- ~Studentas
 - Studentas, [12](#)
- ~Vector
 - Vector< T, Allocator >, [20](#)
- ~Zmogus
 - Zmogus, [29](#)
- addPaz
 - Studentas, [13](#)
- allocator_type
 - Vector< T, Allocator >, [17](#)
- ANTRAS
 - studentai.h, [49](#)
- at
 - Vector< T, Allocator >, [20](#)
- back
 - Vector< T, Allocator >, [20](#)
- begin
 - Vector< T, Allocator >, [21](#)
- capacity
 - Vector< T, Allocator >, [21](#)
- CATCH_CONFIG_MAIN
 - testing.cpp, [58](#)
- cbegin
 - Vector< T, Allocator >, [21](#)
- cend
 - Vector< T, Allocator >, [21](#)
- clear
 - Vector< T, Allocator >, [21](#)
- clearPaz
 - Studentas, [13](#)
- cmpEgz
 - funk.cpp, [34](#)
 - studentai.h, [49](#)
- cmpMedrez
 - funk.cpp, [34](#)
 - studentai.h, [49](#)
- cmpPavarde
 - funk.cpp, [35](#)
 - studentai.h, [49](#)
- cmpRez
 - funk.cpp, [35](#)
 - studentai.h, [50](#)
- cmpVardas
 - funk.cpp, [35](#)
 - studentai.h, [50](#)
- const_iterator
 - Vector< T, Allocator >, [17](#)
- const_pointer
 - Vector< T, Allocator >, [17](#)
- const_reference
 - Vector< T, Allocator >, [17](#)
- const_reverse_iterator
 - Vector< T, Allocator >, [18](#)
- data
 - Vector< T, Allocator >, [21](#), [22](#)
- difference_type
 - Vector< T, Allocator >, [18](#)
- emplace_back
 - Vector< T, Allocator >, [22](#)
- empty
 - Vector< T, Allocator >, [22](#)
- end
 - Vector< T, Allocator >, [22](#)
- erase
 - Vector< T, Allocator >, [22](#), [23](#)
- failgeneravimas
 - funk.cpp, [35](#)
 - studentai.h, [50](#)
- failinputas
 - funk.cpp, [35](#)
 - studentai.h, [50](#)
- failoutputas
 - funk.cpp, [36](#)
 - studentai.h, [50](#)
- fgalunes
 - studentai.h, [53](#)
- front
 - Vector< T, Allocator >, [23](#)
- funk.cpp, [33](#)
 - cmpEgz, [34](#)
 - cmpMedrez, [34](#)
 - cmpPavarde, [35](#)
 - cmpRez, [35](#)
 - cmpVardas, [35](#)
 - failgeneravimas, [35](#)
 - failinputas, [35](#)
 - failoutputas, [36](#)
 - genirasymas, [36](#)
 - inputas, [36](#)
 - logResults, [36](#)
 - lytgen, [36](#)
 - matuoti_perskirstymus, [37](#)
 - operator<<, [37](#)
 - operator>>, [37](#)

- outputas, 37
 - randominputas, 37
 - randompavarde, 38
 - randomvardas, 38
 - rikiuoti, 38
 - skirstyti, 38
 - skirstyti2, 38
 - skirstyti3, 39
 - uzdmat, 39
- fvardai
 - studentai.h, 53
- genirasymas
 - funk.cpp, 36
 - studentai.h, 50
- getEgz
 - Studentas, 13
- getMedrez
 - Studentas, 13
- getPavarde
 - Zmogus, 29
- getPaz
 - Studentas, 13
- getRez
 - Studentas, 14
- getVarDas
 - Zmogus, 29
- info
 - Studentas, 14
 - Zmogus, 29
- inputas
 - funk.cpp, 36
 - studentai.h, 51
- insert
 - Vector< T, Allocator >, 23
- irasymo_failas
 - studentai.h, 53
- irasymo_failas_blogas
 - studentai.h, 53
- irasymo_failas_geras
 - studentai.h, 54
- iterator
 - Vector< T, Allocator >, 18
- logResults
 - funk.cpp, 36
 - studentai.h, 51
- lytgen
 - funk.cpp, 36
 - studentai.h, 51
- main
 - versija_v3.0.cpp, 76
- makeStudent
 - testing.cpp, 58
- matuoti_perskirstymus
 - funk.cpp, 37
 - studentai.h, 51
- max_size
 - Vector< T, Allocator >, 23
- mgalunes
 - studentai.h, 54
- mvardai
 - studentai.h, 54
- my, 9
 - Vector, 9
- operator!=
 - Vector< T, Allocator >, 23
- operator<
 - Vector< T, Allocator >, 24
- operator<<
 - funk.cpp, 37
 - studentai.h, 51
- operator<=
 - Vector< T, Allocator >, 24
- operator>
 - Vector< T, Allocator >, 24
- operator>>
 - funk.cpp, 37
 - studentai.h, 51
- operator>=
 - Vector< T, Allocator >, 25
- operator=
 - Studentas, 14
 - Vector< T, Allocator >, 24
 - Zmogus, 30
- operator==
 - Vector< T, Allocator >, 24
- operator[]
 - Vector< T, Allocator >, 25
- outputas
 - funk.cpp, 37
 - studentai.h, 52
- pavarde
 - Zmogus, 31
- pavardes
 - studentai.h, 54
- PIRMAS
 - studentai.h, 49
- pointer
 - Vector< T, Allocator >, 18
- pop_back
 - Vector< T, Allocator >, 25
- push_back
 - Vector< T, Allocator >, 25
- randominputas
 - funk.cpp, 37
 - studentai.h, 52
- randompavarde
 - funk.cpp, 38
 - studentai.h, 52
- randomvardas
 - funk.cpp, 38
 - studentai.h, 52

rbegin
 Vector< T, Allocator >, 26
refEgz
 Studentas, 14
reference
 Vector< T, Allocator >, 18
refMedrez
 Studentas, 14
refPavarde
 Zmogus, 30
refPaz
 Studentas, 15
refRez
 Studentas, 15
refVardas
 Zmogus, 30
rend
 Vector< T, Allocator >, 26
reserve
 Vector< T, Allocator >, 26
resize
 Vector< T, Allocator >, 26, 27
reverse_iterator
 Vector< T, Allocator >, 18
rikiuoti
 funk.cpp, 38
 studentai.h, 52

setEgz
 Studentas, 15
setPavarde
 Zmogus, 30
setVardas
 Zmogus, 30
shrink_to_fit
 Vector< T, Allocator >, 27
size
 Vector< T, Allocator >, 27
size_type
 Vector< T, Allocator >, 18
skaiciuoti
 Studentas, 15
SkirstymoStrategija
 studentai.h, 49
skirstyti
 funk.cpp, 38
 studentai.h, 52
skirstyti2
 funk.cpp, 38
 studentai.h, 53
skirstyti3
 funk.cpp, 39
 studentai.h, 53
studentai.h, 47
 ANTRAS, 49
 cmpEgz, 49
 cmpMedrez, 49
 cmpPavarde, 49
 cmpRez, 50
 cmpVardas, 50
 failgeneravimas, 50
 failinputas, 50
 failoutputas, 50
 fgalunes, 53
 fvardai, 53
 genirasymas, 50
 inputas, 51
 irasymo_failas, 53
 irasymo_failas_blogas, 53
 irasymo_failas_geras, 54
 logResults, 51
 lytgen, 51
 matuoti_perskirstymus, 51
 mgalunes, 54
 mvardai, 54
 operator<<, 51
 operator>>, 51
 outputas, 52
 pavardes, 54
 PIRMAS, 49
 randominputas, 52
 randompavarde, 52
 randomvardas, 52
 rikiuoti, 52
 SkirstymoStrategija, 49
 skirstyti, 52
 skirstyti2, 53
 skirstyti3, 53
 TRECIAS, 49
 uzdmat, 53
Studentas, 11
 ~Studentas, 12
 addPaz, 13
 clearPaz, 13
 getEgz, 13
 getMedrez, 13
 getPaz, 13
 getRez, 14
 info, 14
 operator=, 14
 refEgz, 14
 refMedrez, 14
 refPaz, 15
 refRez, 15
 setEgz, 15
 skaiciuoti, 15
 Studentas, 12, 13
swap
 Vector< T, Allocator >, 27

TEST_CASE
 testing.cpp, 58–63
testing.cpp, 57
 CATCH_CONFIG_MAIN, 58
 makeStudent, 58
 TEST_CASE, 58–63
TRECIAS
 studentai.h, 49

- uzdmat
 - funk.cpp, [39](#)
 - studentai.h, [53](#)
- value_type
 - Vector< T, Allocator >, [19](#)
- vardas
 - Zmogus, [31](#)
- Vector
 - my, [9](#)
 - Vector< T, Allocator >, [19](#), [20](#)
- Vector< T, Allocator >, [15](#)
 - ~Vector, [20](#)
 - allocator_type, [17](#)
 - at, [20](#)
 - back, [20](#)
 - begin, [21](#)
 - capacity, [21](#)
 - cbegin, [21](#)
 - cend, [21](#)
 - clear, [21](#)
 - const_iterator, [17](#)
 - const_pointer, [17](#)
 - const_reference, [17](#)
 - const_reverse_iterator, [18](#)
 - data, [21](#), [22](#)
 - difference_type, [18](#)
 - emplace_back, [22](#)
 - empty, [22](#)
 - end, [22](#)
 - erase, [22](#), [23](#)
 - front, [23](#)
 - insert, [23](#)
 - iterator, [18](#)
 - max_size, [23](#)
 - operator!=, [23](#)
 - operator<, [24](#)
 - operator<=, [24](#)
 - operator>, [24](#)
 - operator>=, [25](#)
 - operator=, [24](#)
 - operator==, [24](#)
 - operator[], [25](#)
 - pointer, [18](#)
 - pop_back, [25](#)
 - push_back, [25](#)
 - rbegin, [26](#)
 - reference, [18](#)
 - rend, [26](#)
 - reserve, [26](#)
 - resize, [26](#), [27](#)
 - reverse_iterator, [18](#)
 - shrink_to_fit, [27](#)
 - size, [27](#)
 - size_type, [18](#)
 - swap, [27](#)
 - value_type, [19](#)
 - Vector, [19](#), [20](#)
- Vector2.h, [68](#)
- versija_v3.0.cpp, [75](#)
 - main, [76](#)
- Zmogus, [28](#)
 - ~Zmogus, [29](#)
 - getPavarde, [29](#)
 - getVarDas, [29](#)
 - info, [29](#)
 - operator=, [30](#)
 - pavarde, [31](#)
 - refPavarde, [30](#)
 - refVarDas, [30](#)
 - setPavarde, [30](#)
 - setVarDas, [30](#)
 - vardas, [31](#)
 - Zmogus, [29](#)